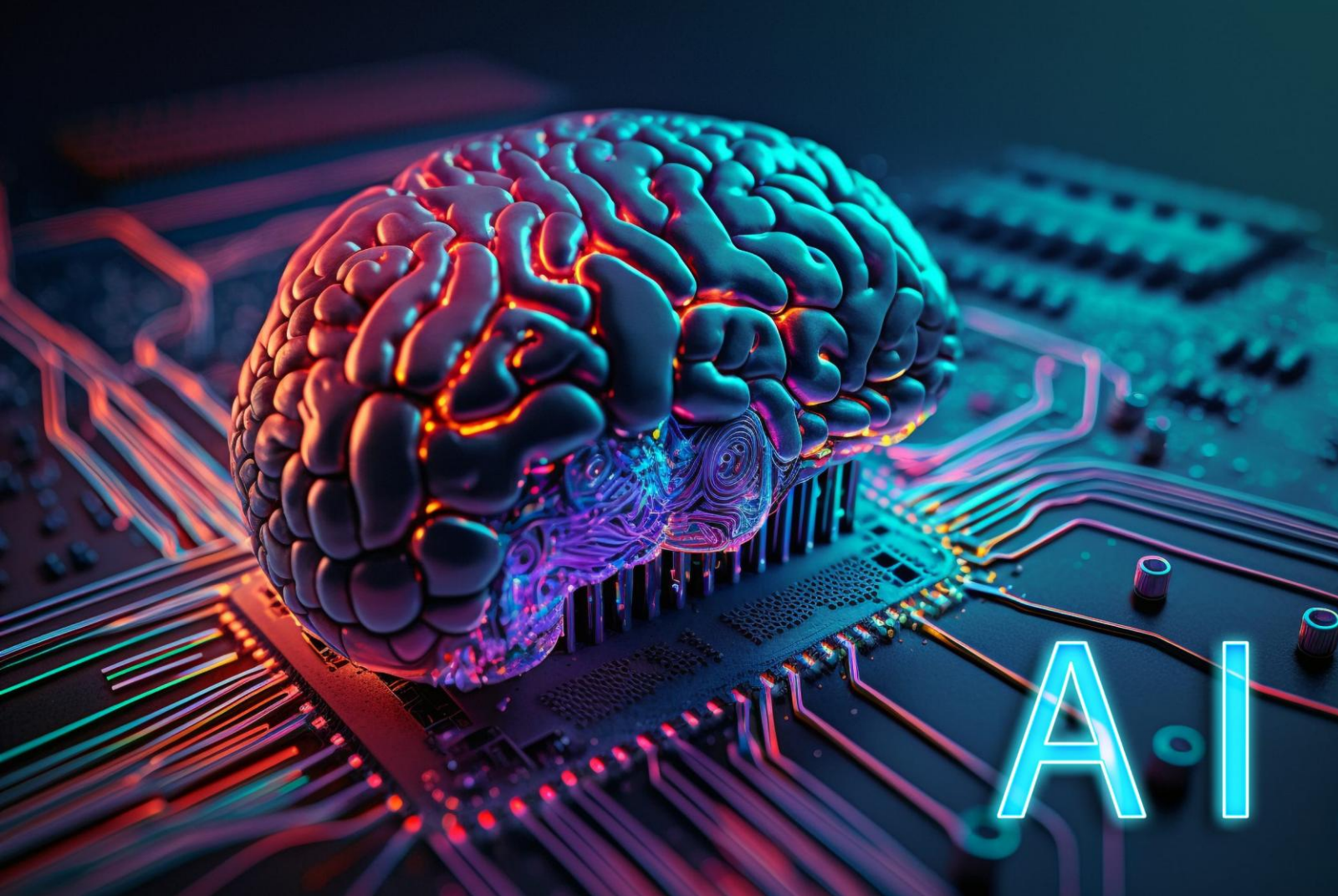




By,

Keerthan Kumar T G

By, Dr. Keerthan, SE, SIT, TUMKUR

Human Intelligence



Any agent who can perform the above is called an Intelligent Agent.

Human Intelligence

- What makes us Humans, intelligent?
- Humans have brains, is that why we are intelligent or do we become intelligent if we learn to use our brains well?

- ☐ When we can give correct answers to people's questions.
- ☐ When we solve a difficult problem or puzzle.
- ☐ When we can decide the correct action to be taken.
- ☐ When we learn something fast.
- ☐ When we are able to recognize many things and remember their names.
- ☐ When we can use clues to guess an answer.

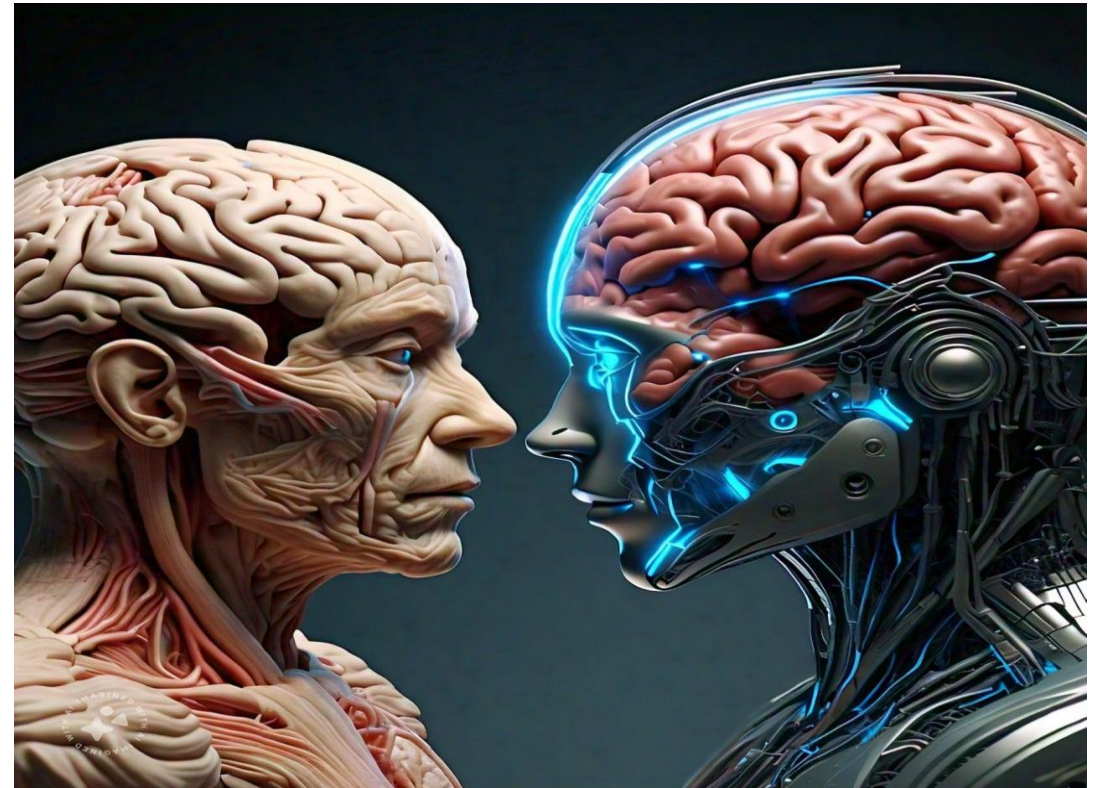
Brain vs. Computer

- In AI, we compare the brain (or the mind) and the computer
 - Our hope: the brain is a *form* of computer.
 - Our goal: we can *create* computer intelligence through programming just as people become intelligent by learning.

But we see that the computer is not like the brain

The computer performs tasks without understanding what its doing

Does the brain understand what its doing when it solves problems?



Brain vs. Computer

•Understanding vs. Execution

- Computer:** Executes tasks based on algorithms and data, but **lacks true understanding**.
- It processes inputs and produces outputs without comprehension of meaning or context.
- Brain:** Engages in problem-solving with an understanding that encompasses emotions, intuition, and context.
- The brain **integrates experiences and knowledge**, allowing for adaptable reasoning.

•Conscious Awareness

- Computer:** Operates through predefined rules and logic, often requiring explicit programming for each task.
- Brain:** Solves problems by utilizing cognitive processes that involve awareness, interpretation, and sometimes subconscious processing.

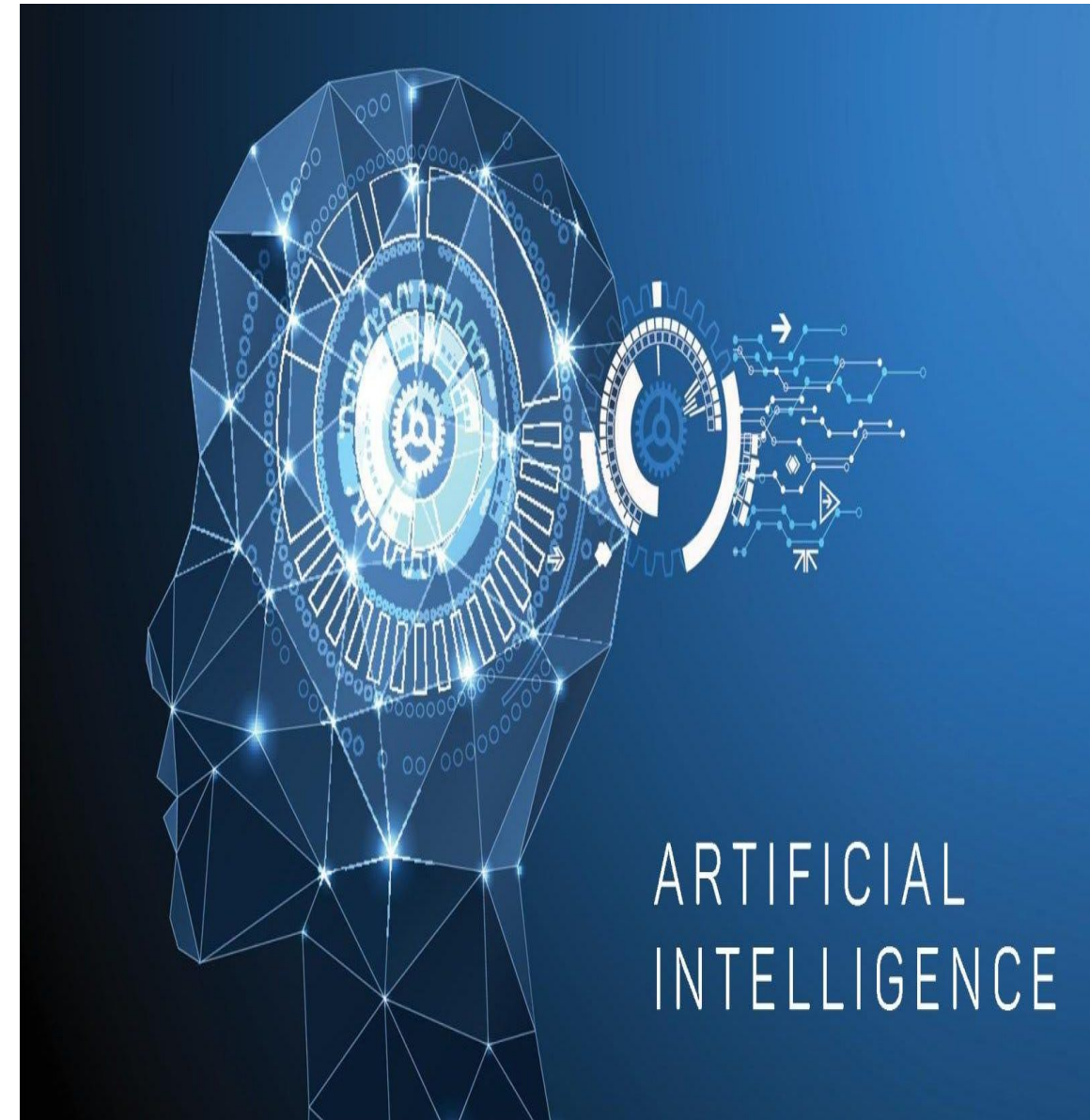
- Learning and Adaptation**

- Computer:** Learns through data and algorithms (in the case of ML), but its "learning" is not akin to human understanding.

- Brain:** Learns through experiences, emotional context, and social interactions, leading to deeper comprehension and insight.

Artificial Intelligence (AI)

- Artificial Intelligence (AI) is a ***broad field of computer science that deals with creating intelligent agents, which are systems that can reason, learn, and act autonomously.*** It aims to simulate human intelligence and problem-solving abilities in machines.



Machine Learning (ML)

- 1.Subset of AI:** Machine learning is a subfield of AI that focuses on enabling ***machines to learn from data and improve their performance over time without explicit programming.***
- 2.Data-Driven:** ML models are trained on large datasets to identify patterns and make predictions or decisions based on new data.
- 3.Techniques Used:** Includes algorithms like supervised learning, unsupervised learning, and reinforcement learning.
- 4.Goal:** The goal of ML is to allow systems to learn autonomously from data and experience.
- 5.Applications:** ML is used in spam email filtering, recommendation engines, image recognition, predictive analytics, and speech recognition.

Relationship Between AI and ML

- **Inclusion:** Machine learning is a part of AI, but not all AI involves machine learning. AI can include *rule-based or logic-based* systems that don't learn from data.
- **Implementation:** AI can be achieved using ML techniques, but there are *other ways to implement AI without machine learning*.
- **Evolution:** ML has become the most popular method for achieving AI due to its ability to handle complex tasks and learn from large datasets.

There are several ways to implement AI without relying on machine learning.

- 1.Rule-Based Systems:** These systems use predefined rules to make decisions. Expert systems, like *medical diagnosis programs*, often rely on a set of if-then rules.
- 2.Logic Programming:** This involves using *formal logic* to represent facts and rules about some problem domain. Prolog is a common language used for this purpose.
- 3.Search Algorithms:** AI can be implemented through algorithms that explore *problem spaces*, such as depth-first search or A* for pathfinding.
- 4.Constraint Satisfaction:** This involves solving problems by defining *constraints and finding solutions* that satisfy those constraints, often used in scheduling and resource allocation.

There are several ways to implement AI Contd.

5. Genetic Algorithms: Inspired by the process of *natural selection*, these algorithms use techniques such as mutation and crossover to solve optimization problems without needing training data.

6. Fuzzy Logic: This approach deals with reasoning that is approximate rather than fixed and exact. It can be useful for handling uncertainty and imprecision in decision-making.

7. Heuristic Methods: These are strategies derived from previous experiences with similar problems, providing shortcuts to find solutions faster than traditional methods.

8. Symbolic AI: This approach represents knowledge in a form that machines can manipulate, often using ontologies and knowledge graphs.

9. Natural Language Processing (NLP) without ML: Techniques like rule-based parsing or regular expressions can be used for simpler language tasks without machine learning.

Why is AI required?

- Efficiency and productivity:** AI can automate tasks that are repetitive, time-consuming, or error-prone, leading to increased efficiency and productivity.
- Problem-solving:** AI can *help solve complex problems* that humans may struggle with, such as *medical diagnosis, climate modeling, and financial forecasting*.
- Decision-making:** AI can provide valuable *insights and recommendations* for decision-making in various fields, including business, healthcare, and government.
- Innovation:** AI is driving *innovation in many industries*, leading to the development of new products and services.
- Personalization:** AI can *personalize experiences for individuals*, such as recommending products, movies, or music based on their preferences.

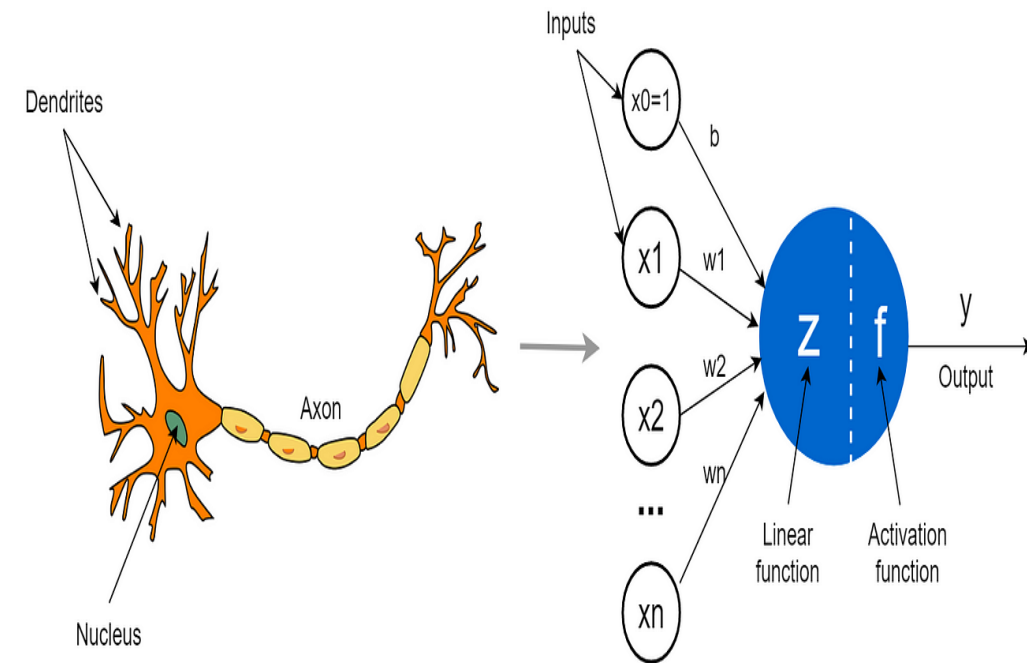
What is AI?

- Artificial Intelligence (AI) refers to the **simulation of human intelligence in machines** that are programmed to **think, learn, and perform tasks** that typically require **human cognition**.
- There are several types of AI, ranging from **narrow AI**, which is designed to handle a specific task (like facial recognition or voice assistants), to **general AI**, which has the potential to perform any intellectual task that a human can do.



Human cognition

- Human cognition refers to the *mental processes* involved in *acquiring knowledge and understanding* through *thought, experience, and the senses*.
- It encompasses a wide range of functions, including *perception, attention, memory, learning, reasoning, problem-solving, decision-making, and language*.
- These cognitive processes allow humans to *interpret and interact with their environment*, make sense of complex information, and adapt to new situations.
- Cognitive abilities can vary from person to person and can be influenced by factors such as *genetics, environment, education, and experience*.



What is AI?

Thinking Humanly

“The exciting new effort to make computers think ... *machines with minds*, in the full and literal sense.” (Haugeland, 1985)

“[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ...” (Bellman, 1978)

Acting Humanly

“The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990)

“The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)

Thinking Rationally

“The study of mental faculties through the use of computational models.”
(Charniak and McDermott, 1985)

“The study of the computations that make it possible to perceive, reason, and act.”
(Winston, 1992)

Acting Rationally

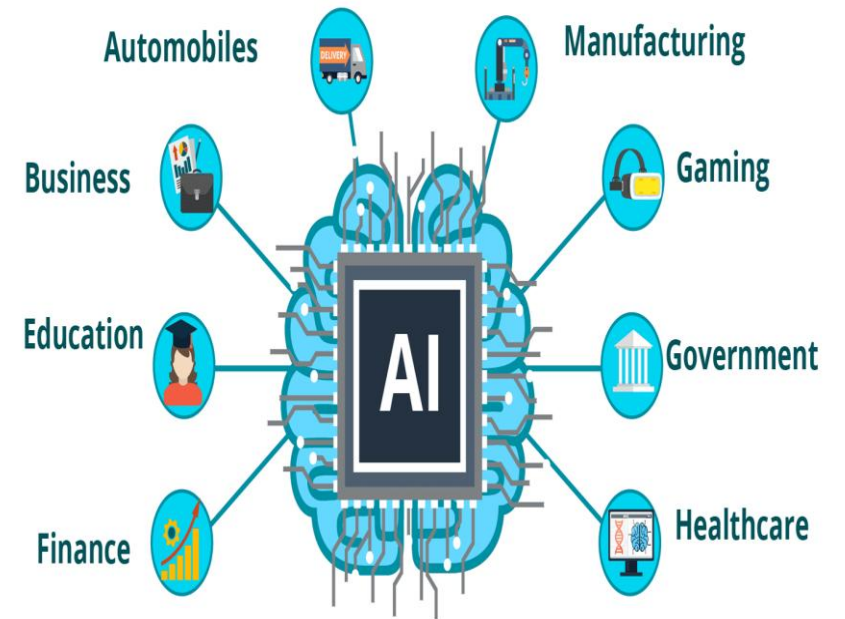
“Computational Intelligence is the study of the design of intelligent agents.” (Poole *et al.*, 1998)

“AI ...is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)

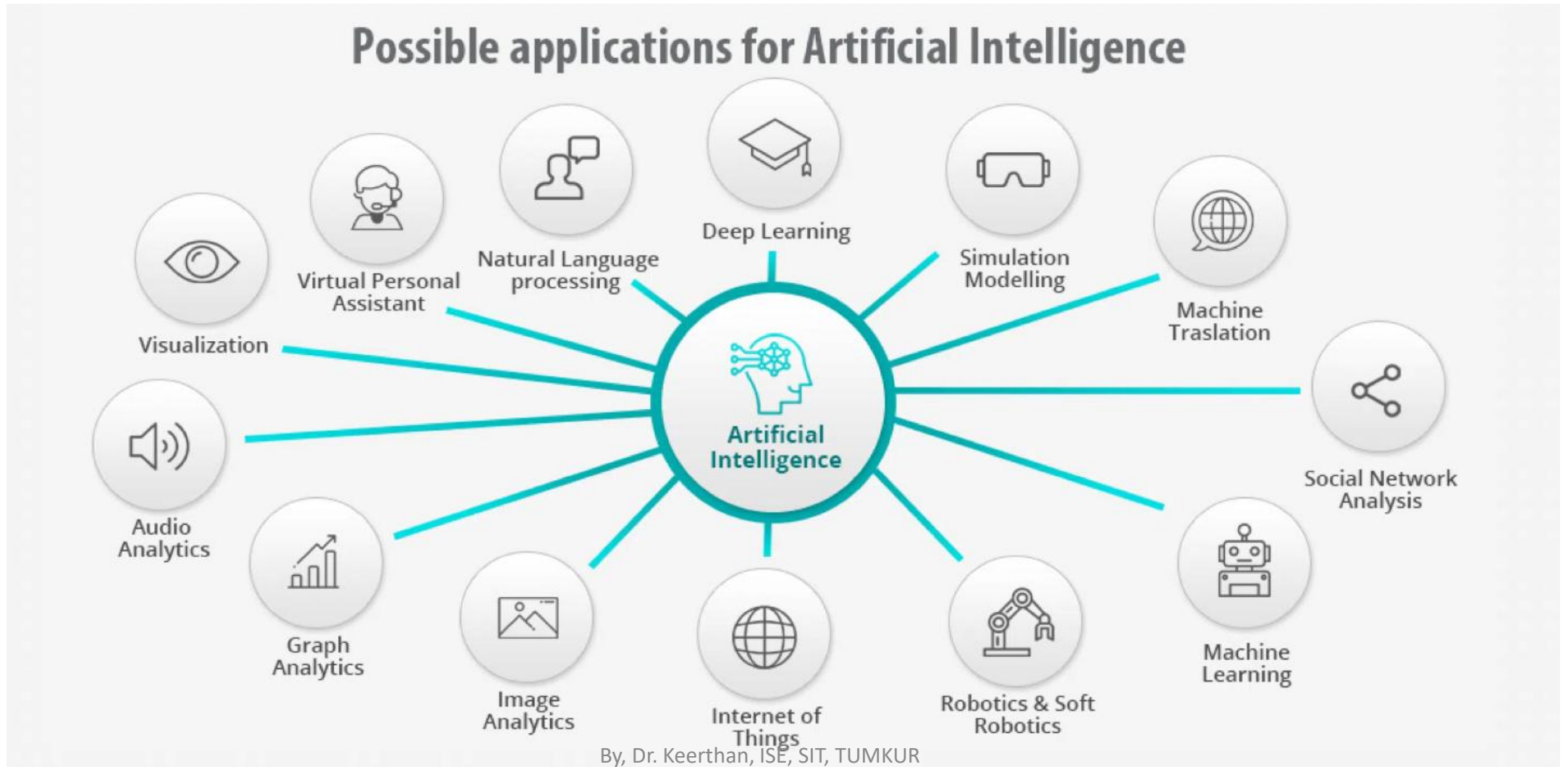


AI applications

- AI currently encompasses a huge variety of subfields,
 - Playing chess
 - Proving mathematical theorems
 - Writing poetry
 - Driving a car on a crowded street
 - Healthcare -Diagnosing diseases
 - Machine learning
 - Natural language processing
 - Robotics
 - Computer vision
 - Finance
 - Education
 - Autonomous systems.
- AI is relevant to any intellectual task; it is truly a universal field.



AI applications



Acting humanly: The Turing Test approach

- The Turing Test, proposed by British mathematician and logician **Alan Turing** in 1950, is a concept within the realm of Artificial Intelligence (AI) that addresses the question, "**Can machines think?**".
- A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.
- The test is designed to evaluate a machine's ability to exhibit intelligent behavior that is indistinguishable from that of a human.
- Turing Test remains an important historical concept in AI, symbolizing the challenge of creating machines that can not only perform tasks but also engage in meaningful, human-like interactions.

How does it work?

- A human evaluator communicates with two entities: one is a human and the other is a machine.
- The evaluator asks questions to both entities and receives their responses.
- If the evaluator cannot reliably distinguish between the human and the machine after multiple interactions, it can be concluded that the machine has passed the Turing Test.

Implications of passing the Turing Test:

- If a machine can pass the Turing Test, it would suggest that it has achieved a level of intelligence that is indistinguishable from human intelligence.
- This would have significant implications for various fields, *including artificial intelligence research, philosophy, and ethics.*

Turing Test

- So that programming a computer to pass a rigorously applied test provides plenty to work on.
- The computer would need to possess the following capabilities:
 - **Natural language processing** : to understand and generate human language;
 - **Knowledge representation** to store what it knows or hears;
 - **Automated reasoning** to use the stored information to answer questions and to draw new conclusions
 - **Machine learning** to adapt to new circumstances and to detect and extrapolate patterns.
 - **Computer vision** to perceive objects.
 - **Robotics** to manipulate objects and move about.
- These six disciplines compose most of AI, and Turing deserves credit for designing a test that remains relevant

Example

- **Set-up:**
- An **interrogator (human judge)** communicates through a terminal.
- Two participants:
 - **A:** a human
 - **B:** a machine
- The judge can ask **any questions** and, after a period of interaction, must decide which participant is the human.
- **Example:**

A chatbot engages in open-ended conversation, answering questions like “*Explain photosynthesis*”, “*Tell me a joke*”, or “*What’s 17×19?*”. It can also ask relevant follow-up questions.
- If, across multiple judges and sessions, the judge cannot identify the machine better than random guessing, the system is considered to have **passed the Turing Test**.
- .

Thinking humanly: The cognitive modeling approach

- The **cognitive modeling approach** is a method in artificial intelligence (AI) aimed at mimicking human thought processes.
- It seeks to understand how humans think and to develop models that replicate these processes in machines.
 - **Definition:** Cognitive modeling involves creating computational models that simulate human cognitive functions such as *reasoning, problem-solving, memory, and decision-making*.
 - **Relation to AI:** AI systems developed using the cognitive modeling approach aim to replicate the way humans process information. This is different from simply creating systems that act intelligently.

The cognitive modeling approach contd.

- Key Models of Cognition:
 - **Symbolic models:** Represent human knowledge using symbols and rules (e.g., production systems, logic).
 - **Connectionist models (Neural networks):** Model human thinking as networks of simple units (similar to neurons in the brain).
 - **Hybrid models:** Combine elements of both symbolic and connectionist approaches.

The cognitive modeling approach contd.

- To claim a program thinks like humans, we must understand how humans think.
- Methods:
 - introspection (self-observation of thoughts),
 - psychological experiments (observing actions),
 - brain imaging (observing brain activity).
- Applications:
 - **Cognitive Science combines AI models and psychology experiments to develop testable theories of the mind.**

The cognitive modeling approach contd.

- **Challenges:**

- Understanding human cognition is extremely **complex**, and **no single model fully captures** all aspects of human thought.
- Models can be computationally expensive and difficult to scale for real-world applications.

- **Importance for AI:**

- Studying human thinking helps develop more intuitive and efficient AI systems.
- It promotes advancements in fields like Human-Computer Interaction (HCI), where systems need to understand and predict user behavior.

Thinking rationally: The “laws of thought” approach

- Laws of Thought: The Logician Approach:
- **Aristotle** introduced ***syllogisms*** (structured reasoning rules) for deriving correct conclusions from correct premises (e.g., “All men are mortal; Socrates is a man; therefore Socrates is mortal”).
- This led to the field of **logic**, which formalizes reasoning processes.
- By the 19th century, logicians developed precise notation for representing statements about objects and their relations.
- By the 1960s, AI programs existed that could, in principle, solve any problem expressed in logical notation.
- The **logician** tradition in AI seeks to build intelligent systems based on logical reasoning.

Example

1. Syllogism (Aristotle's reasoning pattern)

Example (Shopping):

- Premise 1: All electronic items in this shop have a 1-year warranty.
- Premise 2: This laptop is an electronic item from this shop.
- Conclusion: Therefore, this laptop has a 1-year warranty.

2. Logic (general reasoning rules)

Example (Medical diagnosis):

- Rule 1: If a patient has fever AND cough, then the patient may have the flu.
- Rule 2: If a patient has rash AND fever, then the patient may have measles.
- Fact: Patient has fever and cough.
- Conclusion: Patient may have flu.

3. Logicist (AI approach based on logic)

Example (Route planning in Google Maps):

- Knowledge in logic:
 - "If there is a traffic jam on Road A, then avoid Road A."
 - "If Road B is open AND shorter, then take Road B."
- Input facts: Road A = jammed, Road B = clear.
- AI system applies logical rules → Suggests Road B.

- **Challenges:**

- Difficult to express informal/uncertain knowledge in strict logical terms.
- Computational limits: even small problems can overwhelm computers without efficient strategies.

Acting Rationally: The Rational Agent Approach

- In the field of Artificial Intelligence (AI), **acting rationally** refers to the ability of a system or agent to make decisions and take actions that **maximize its chances of achieving specific goals**.
- The **rational agent approach** focuses on building systems that **behave optimally** in any given situation based on the information they have.
 - A **rational agent** is one that chooses actions that are expected to maximize its performance measure, given the available knowledge and percepts
 - **Perception, Decision, and Action:**
 - A rational agent **perceives** its environment through sensors (inputs like cameras, microphones, etc.), processes the information, makes **decisions** based on its goals and knowledge, and takes **actions** using effectors (outputs like motors or commands).
 - The agent must consider the future impact of its actions and choose the best course that leads to the most favorable result.

Characteristics of a Rational Agent:

- Autonomous:** Acts independently, using its perception to take actions without constant human intervention.
- Goal-oriented:** Aims to achieve specific objectives, acting in a way that is expected to move towards those goals.
- Adaptive:** Can update its knowledge and behavior based on new information.
- Interactive:** Interacts with its environment to gather data, understand changes, and adjust its actions.

The Rationality of Actions:

- An agent's actions are rational if:
 - It chooses actions that **maximize the likelihood of success based on the knowledge it has.**
 - It takes into account the **uncertainty** of the environment and potential outcomes.

Types of Rational Agents:

- **Simple reflex agents:** React to their environment based on predefined rules, without considering the future.
- **Goal-based agents:** Act based on goals, considering future outcomes to determine the best action.
- **Utility-based agents:** Evaluate different possible outcomes and choose actions that maximize their expected utility (a measure of satisfaction).
- **Learning agents:** Improve their performance over time by learning from past experiences.

1. Intelligent Agent (IA)

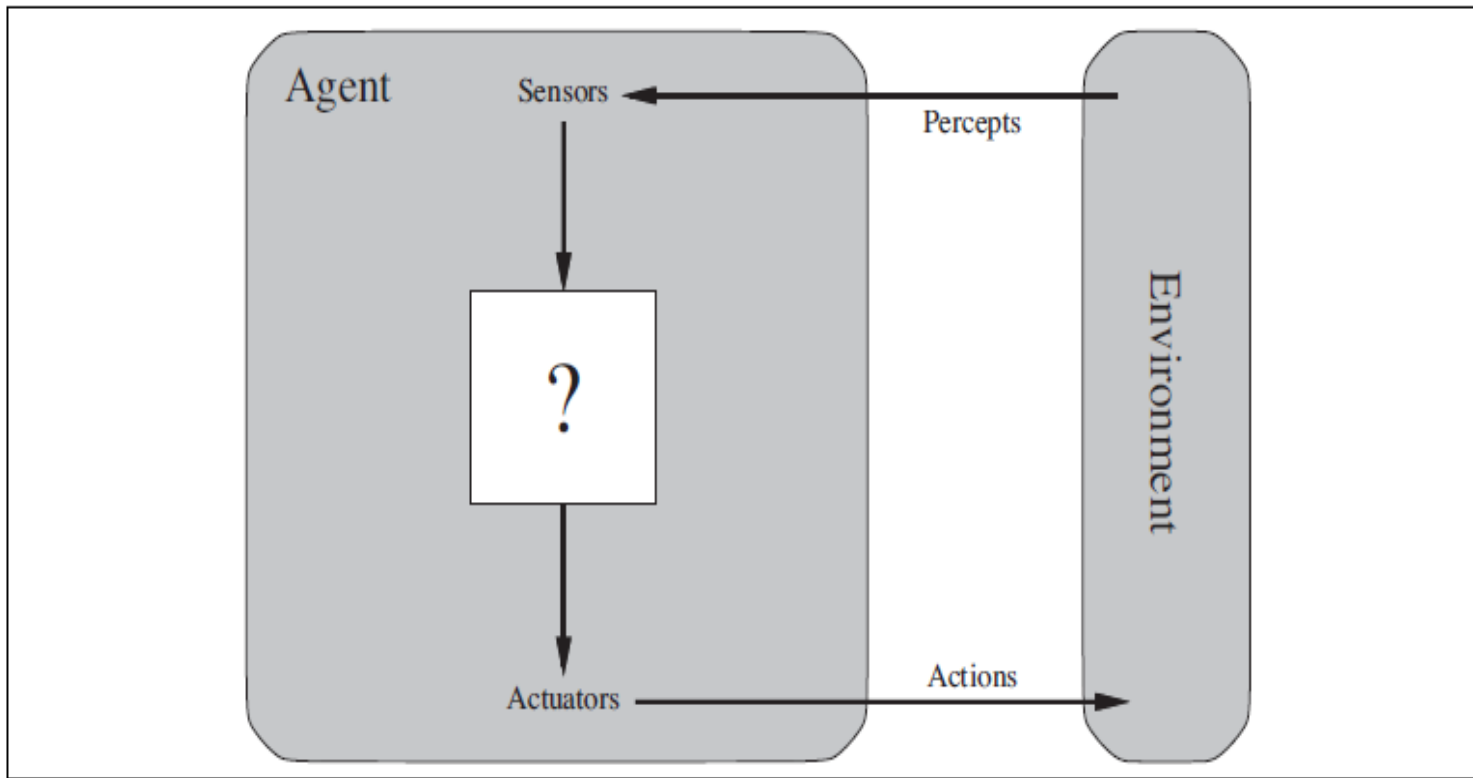
- **Definition:** An agent is intelligent if it can perceive its environment, learn from it, and act to achieve its goals.
 - **Key idea:** Intelligence = ability to **learn, adapt, and make decisions**.
 - **Example:**
 - A **virtual assistant** (like Siri or Alexa) that learns user preferences and improves responses.
 - A **robot vacuum cleaner** that maps the room and updates its cleaning path.
-

2. Rational Agent (RA)

- **Definition:** A rational agent is one that **chooses the action that maximizes its performance measure**, based on available knowledge and percepts.
- **Key idea:** Rationality = **doing the right thing** (maximizing success, not necessarily “being intelligent”).
- **Example:**
 - A **chess program** that always plays the best possible move given its knowledge of the board.
 - A **self-driving car** taking the **safest, shortest path** to reach the destination.

INTELLIGENT AGENTS

- Agents and Environments: An **agent** is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.



- Sensors like cameras and infrared detectors allow the robot to "see" obstacles.
- Actuators like wheels or robotic arms enable the robot to move around or pick up objects.

Figure:1 Agents interact with environments through sensors and actuators.

AGENTS Contd.

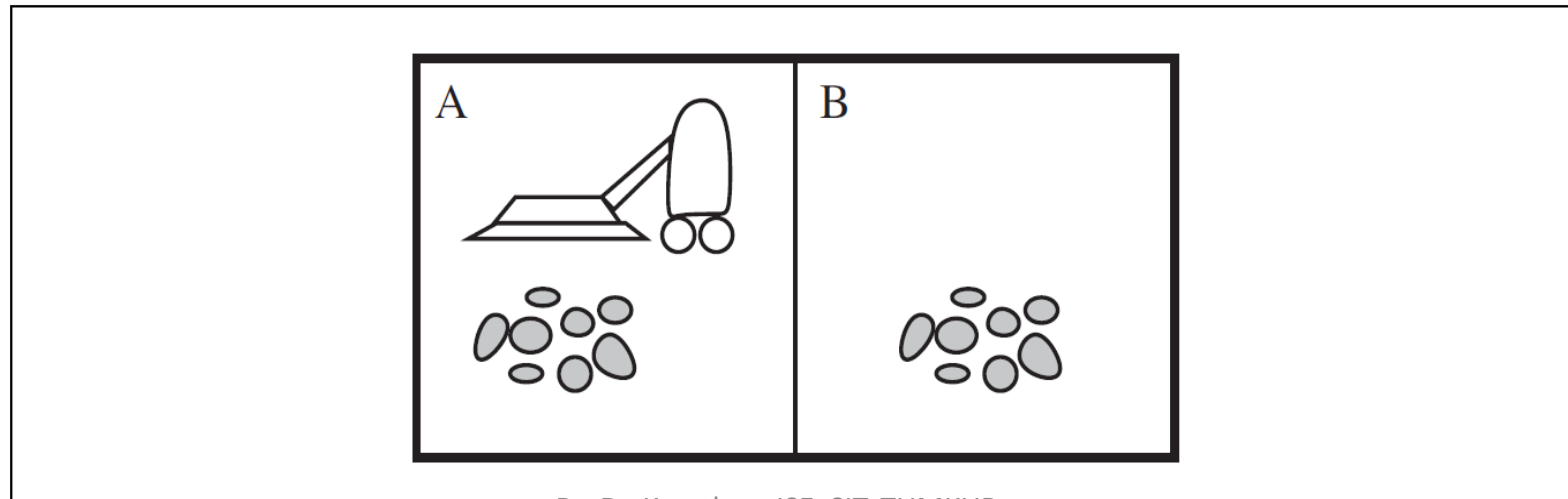
- A human agent **has eyes, ears**, and other organs for **sensors** and **hands, legs, vocal tract**, and so on for **actuators**.
- A **robotic agent** might have *cameras and infrared range finders* for sensors and **various motors for actuators**.
- A **software agent** *receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets*

AGENTS Contd.

- We use the term **percept** to refer to the agent's perceptual inputs at any given instant. An agent's **percept sequence** is the complete history of everything the agent has ever perceived.
- In general, an agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived.
- **Agent Program**
 - An agent program is the core software or set of instructions that controls an agent's behavior. It takes the agent's perceptions from the environment (through sensors) and processes that information to make decisions, which it then converts into actions (via actuators).
- **Agent Function**
 - An agent function is a mathematical or conceptual mapping that describes how an agent chooses actions based on its percept sequence (the history of all its perceptions).
 - It defines the behavior of the agent by specifying the action to take in response to a given sequence of percepts.
 - In essence, it maps percepts (inputs from the environment) to actions (outputs).

To illustrate these ideas, we use a very simple example: **vacuum-cleaner**

- the vacuum-cleaner world shown in Figure 2.
- This world is so simple that we can describe everything that happens; it's also a made-up world, so we can invent many variations. *This particular world has just two locations: squares A and B.*
- The vacuum agent perceives which square it is in and whether there is dirt in the square.
- It can choose to move left, move right, suck up the dirt, or do nothing.
- **One very simple agent function is the following: if the current square is dirty, then suck; otherwise, move to the other square.**



By, Dr. Keerthana, ISE, SIT, TUMIKUR

Figure 2. A vacuum-cleaner world with just two locations.

example: vacuum-cleaner

- Looking at Figure 2.3, we see that various vacuum-world agents can be defined simply by filling in the right-hand column in various ways.

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>
<i>[A, Clean], [A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

Figure 3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.

example: vacuum-cleaner contd.

- The obvious question, then, is this: *What is the right way to fill out the table?* In other words, what makes an agent good or bad, intelligent or stupid?
- If the agent uses some randomization to choose its actions, then we would have to **try each sequence many times to identify the probability of each action.**
- One might imagine that **acting randomly** is rather silly, but we show later in this chapter that it can be very intelligent.

GOOD BEHAVIOR: THE CONCEPT OF RATIONALITY

- **Rationality**
- **Omniscience**
- **Learning**
- **Autonomy**

Rationality

- **Rationality** refers to the ability of an agent to make decisions and take actions that **maximize its performance** based on the information it has and the goals it seeks to achieve.
- A rational agent acts in a way that ensures the *best possible outcome in any given situation*, considering the limitations of the environment and its own knowledge.
- What is rational at any given time depends on four things:
 - The *performance measure* that defines the criterion of success.
 - The agent's *prior knowledge* of the environment.
 - The *actions* that the agent can perform.
 - The agent's *percept sequence* to date.

Omniscience:

- **Omniscience** refers to having *complete and perfect knowledge* of all possible outcomes and the environment.
- An omniscient agent would know everything about **the past, present, and future, including the consequences** of all possible actions.
- **Infeasibility:** In real-world scenarios, omniscience is impossible because agents cannot have full knowledge of every aspect of their environment. They may not know all relevant factors or the precise outcomes of actions in advance.
- **Rationality vs. Omniscience:** Rational agents make decisions based on the information available at the time, even if they don't know everything. Rationality doesn't require omniscience—it only requires making the best decisions based on the agent's knowledge and resources.

- Example:
- A chess-playing agent cannot foresee all future moves of its opponent but uses available data to make rational choices based on probability and strategy.

Learning

- **Learning** allows an agent to *improve its performance over time by gaining new knowledge from its experiences*. A **learning agent** can adapt its behavior to become more effective in achieving its goals.
- **Key Components:**
 - **Learning Element:** *Enables the agent to modify its behavior based on past experiences.*
 - **Performance Element:** *Uses the knowledge gained to make decisions and take actions.*
 - **Critic:** Provides feedback on the *agent's actions*, allowing the learning process to adjust and improve behavior.
 - **Problem Generator:** Helps the agent *explore new actions or strategies* that might lead to better solutions.
- **Importance:** Learning is essential for agents that operate in complex or changing environments, where predefined rules or knowledge are insufficient.

- **Example:**
- A **self-driving car** learns from its driving history, improving its ability to navigate roads, avoid obstacles, and adapt to new environments.

Autonomy:

- **Autonomy** refers to an *agent's ability to operate independently*, making decisions and taking actions without human intervention.
- An **autonomous agent** can control its own behavior by using its percepts and knowledge to achieve its objectives.
- **Level of Autonomy:** Autonomy can range from fully controlled by humans to *fully independent agents*. Higher autonomy means the agent requires less human input and can adapt on its own.
- **Adaptive Behavior:** An autonomous agent can **adapt to new situations** and environments based on its own experiences and learned knowledge.
- **Balance with Learning:** As an *agent learns more about its environment, it can become more autonomous*, relying less on predefined rules and more on its experiences.

Autonomy contd.

- **Example:**

- A **robotic vacuum cleaner** operates autonomously by sensing obstacles, learning the layout of the room, and adjusting its cleaning patterns without human guidance.

Specifying the Task Environment

- The rationality of the simple vacuum-cleaner agent, we had to specify the **performance measure, the environment, and the agent's actuators and sensors.**
- We group all these under the heading of the **task environment**. For the acronymically minded, we call **PEAS** this the **PEAS** (Performance, Environment, Actuators, Sensors) description.
- PEAS description for the taxi's task environment:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Examples of agent types and their PEAS descriptions.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Properties of task environments

- Task environments in artificial intelligence (AI) are categorized based on different properties that help define the nature of the tasks or problems being solved. These properties influence how agents interact with their environments.
- **Deterministic vs. Stochastic**
- **Deterministic Environment:** The outcome of an agent's action is predictable and certain. There is no randomness involved.
 - **Example: Chess** is a deterministic environment. The movements of pieces are fixed, and given a certain board configuration and action, the next board state is completely predictable. There is no uncertainty in how the pieces move.
- **Stochastic Environment:** The outcome of an action may involve randomness or uncertainty, meaning an action can result in different outcomes.
 - **Example: Self-driving cars** operate in a stochastic environment. While the car may plan to move straight, an unexpected event (like a pedestrian suddenly crossing) can cause the environment to change in unpredictable ways.

Properties of task environments Contd.

- **Fully Observable vs. Partially Observable**
- **Fully Observable Environment:** The agent has access to all relevant information about the environment at all times. No hidden information exists.
 - **Example:** In **tic-tac-toe**, the entire game state is visible to both players at all times. There's no hidden information, so it's fully observable.
- **Partially Observable Environment:** The agent does not have access to the complete state of the environment. There may be hidden or inaccessible information.
 - **Example:** In **poker**, players can only see their own cards, while the other players' cards are hidden. This makes poker a partially observable environment.

Properties of task environments Contd.

- **Known vs. Unknown**
- **Known Environment:** The agent knows the rules of the environment and how actions affect the state of the environment. The models of how the environment behaves are known in advance.
 - **Example: A robot navigating a factory floor** where the layout, obstacles, and possible actions are predefined and known. The robot can rely on its pre-programmed knowledge to make decisions.
- **Unknown Environment:** The agent does not know how its actions will affect the environment, and it must learn through exploration or experimentation.
 - **Example: A robot exploring a new, uncharted planet** has to learn the rules of the environment, such as how terrain types affect movement or where resources might be found.

Properties of task environments Contd.

- **Static vs. Dynamic**
- **Static Environment:** The environment remains unchanged while the agent is deliberating or performing actions. The world does not change on its own.
 - **Example: Solving a jigsaw puzzle** is a static environment. The puzzle does not change or modify itself while the agent (person) is working on it. It stays the same until the agent moves a piece.
- **Dynamic Environment:** The environment can change while the agent is deliberating or executing actions, even without the agent's involvement.
 - **Example: Traffic systems** are dynamic environments. The traffic lights, pedestrians, and other vehicles continuously change even if the self-driving car has not made a move yet. This requires the car to continuously adapt to the changing environment.

Properties of task environments Contd.

- **Single-Agent vs Multi-Agent**
- **Single-Agent Environment**
- **Definition:** In a single-agent environment, there is only one agent acting or making decisions, and the environment responds solely to that agent's actions. The agent does not have to consider the influence or behavior of any other agents.
- **Example:**
 - **Solving a maze:** In a maze-solving scenario, a single agent (e.g., a robot) is navigating through the maze. The robot's actions (moving forward, turning left, etc.) only affect its own progress, and it doesn't need to account for other agents. The environment (maze walls) doesn't change based on any other entities.
- **Key Consideration:** The agent is only concerned with optimizing its own performance in achieving its goal, without competition or cooperation.

Properties of task environments Contd.

- **Multi-Agent Environment**
- **Definition:** In a multi-agent environment, there are multiple agents that interact with each other. The actions of one agent can affect the other agents, and the environment's outcome depends on the collective actions of all agents involved. Agents may either compete, cooperate, or both.
- **Types of Multi-Agent Interactions:**
 - **Competitive:** Agents are working against each other, trying to maximize their own success at the expense of others.
 - **Cooperative:** Agents work together to achieve a shared goal.
- **Example:**
 - **Competitive: Chess** is a classic example of a multi-agent competitive environment. Two players (agents) are playing against each other, and each agent's decisions directly impact the other's strategy. The goal is to defeat the opponent by checkmating their king.
 - **Cooperative: Robot soccer** involves multiple agents (robots) on a team working together to score goals against another team of agents. The agents must collaborate to pass, defend, and score effectively.

Contd.

- **Episodic vs. sequential**
- **Episodic Environment**
- **Definition:** In an episodic environment, an agent's experience is divided into distinct, independent episodes. Each episode consists of a single perception-action sequence, and the outcome of one episode does not affect the next one. The agent does not need to worry about long-term consequences of its actions.
- **Example:**
 - **Image classification:** In a task where an agent is classifying images (e.g., identifying if an image contains a cat), each image is an independent episode. The decision made on one image does not influence future images or the outcomes of other classifications.
- **Key Consideration:** Since actions in each episode are independent of previous or future actions, agents can focus solely on maximizing their performance for the current episode without considering long-term strategy or history.

Properties of task environments Contd.

- **Sequential Environment**
- **Definition:** In a sequential environment, the current action of the agent affects not only the immediate outcome but also influences future situations or actions.
- There is a continuous interaction where the agent must consider the long-term consequences of its decisions.
- **Example:**
 - **Chess:** Chess is a sequential environment because the move you make on one turn affects the state of the board for future moves. The agent (player) needs to plan multiple moves ahead and think strategically about how each action will impact the overall game.
- **Key Consideration:** In sequential environments, agents must account for the future impact of their current actions, making it necessary to plan ahead and consider how current actions will influence future states and goals.

Properties of task environments Contd.

- **Continuous VS Discrete**
- **Continuous Environment**
- **Definition:** In a continuous environment, the state, actions, and/or time can take on a continuous range of values. The environment changes smoothly over time, and the agent's actions can be any value from a range rather than a set of distinct choices.
- **Example:**
 - **Self-driving car:** A self-driving car operates in a continuous environment. The car can move in any direction at varying speeds, and its position, speed, and steering angle can take on a continuous range of values. The environment (road, obstacles) changes fluidly as time progresses.
- **Key Consideration:** Continuous environments are more complex to model because they require handling an infinite number of potential states and actions, often requiring calculus, differential equations, or approximation methods.

Properties of task environments Contd.

- **Discrete Environment**
- **Definition:** In a discrete environment, the states, actions, and/or time are distinct and countable. There is a limited number of possible states and actions, and the agent operates in clearly defined steps.
- **Example:**
 - **Chess:** Chess is a discrete environment where there are a finite number of states (board configurations) and a finite set of possible actions (legal moves). The game proceeds in turns, and each move leads to a discrete change in the state of the game.
- **Key Consideration:** Discrete environments are often easier to model computationally because the agent operates within a set of well-defined choices and states.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Examples of task environments and their characteristics.

The Structure of Agents

- The structure of an agent refers to its architecture and the decision-making process that enables it to interact with its environment.
- The job of AI is to design an **agent program** that implements the agent function the mapping from percepts to actions. We assume this program will run on some sort of computing device with physical sensors and actuators, we call this the **architecture**:
 - *agent = architecture + program*

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

This algorithm defines a very simple type of agent: the **table-driven agent**.

It works by storing **all possible percept sequences** (everything the agent has ever sensed) in a **table** and then deciding an action by looking up the sequence in that table.

percepts:

A sequence (list) that keeps track of everything the agent has ever perceived. Initially, it is empty.

table:

A pre-defined lookup table that contains entries mapping percept sequences → actions.

(e.g., if the agent has perceived [clean, dirty], the table specifies the action like "suck" or "move").

Steps

Append percept to percepts

Each time the agent perceives something new from the environment, it is added to the history list percepts.

Lookup action in the table

The agent looks up the full percept sequence in the table to decide what action to take.

Return the action

The chosen action is executed.

Example: First percept: [RoomA = dirty] → lookup in table → action: "suck".

Second percept: [RoomA = dirty, RoomA = clean] → lookup → action: "move to RoomB".

- Lookup table:

- Let's agent is having two sensors.

Location	Dirt
2 Possible values A or B	2 Possible values dirty or clean

- $2 * 2 = 4$ precepts at each time stamp.

A clean
A dirt
B clean
B dirt

- Percept is appended to percepts.

- i.e. Percept at $t1 = [A, CLEAN]$

- Percepts = ([A, CLEAN])

- Next Percept at $t2 = [B, CLEAN]$

- Percepts = ([A, CLEAN], [B, CLEAN]) i.e., Sequence of percepts

- For each time the percepts come that will be added to the percepts sequence. i.e. History.

Lookup table

Percept Sequence	Corresponding Action
[A, clean]	Move right
[A, dirt]	suck
[B, clean]	Move left
[B, dirt]	suck

Percept Sequence	Corresponding Action
[A, clean]	Move right
[A, dirt]	suck
[B, clean]	Move left
[B, dirt]	suck
[A, clean], [A, clean]	Move right
[A, clean], [A, dirt]	suck
[A, clean], [B, clean]	Move left
[A, clean], [B, dirt]	suck

At time t1 four percepts, therefore size of the table =4 entries.

Next at time t2, for each four entries we can again four different possibilities.

EX: for [A, clean], we have 4 different possibilities.

Total Entries in a table at t2
 $=4+4*4=4+16=4+4^2$

Total Entries in a table at t3
 $=4+16+16*4=4+16+64=4+16+4^3$

Let P be the set of possible percepts and let T be the lifetime of the agent (the total number of percepts it will receive). The lookup table will contain entries.

$$\sum_{t=1}^T |P|^t$$

The Structure of Agents contd.

- Agents can be categorized based on their complexity and design, typically falling into one of the following categories:
- Four basic kinds of agent programs that embody the principles underlying almost all intelligent systems:
 - **Simple Reflex Agents**
 - **Model-based reflex agents**
 - **Goal-based agents**
 - **Utility-based agents**

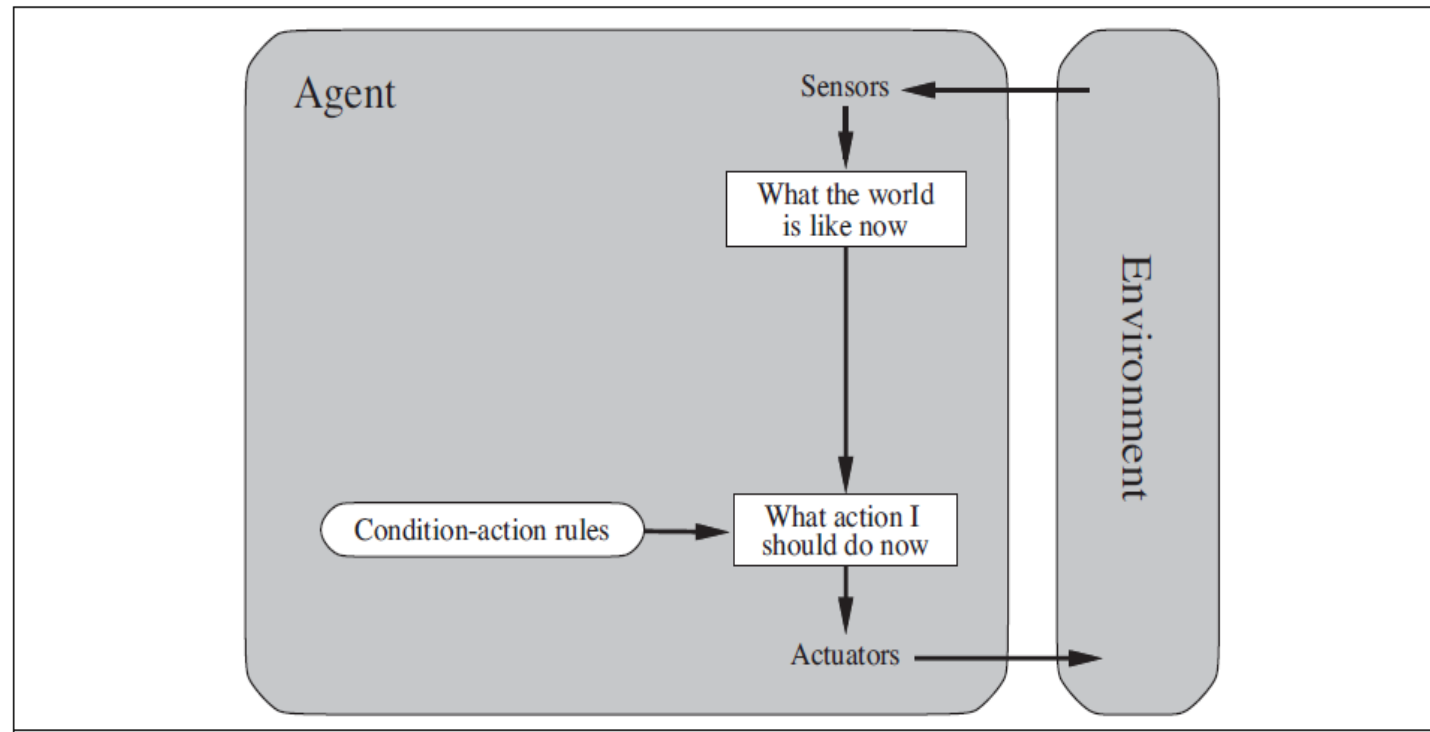
Simple Reflex Agents

- **Simple Reflex Agents:** These agents act purely based on the current percept, ignoring the history of past actions.
- They follow *condition-action rules* (if-then statements) and respond immediately to the environment.
- **Example:** A *thermostat* that turns on the heater if the temperature falls below a certain threshold.
- These agents select actions on the basis of the current percept, ignoring the rest of the percept history.

Simple Reflex Agents

The car in front is braking.” Then, this triggers some established connection in the agent program to the action “initiate braking.” We call such a connection a **condition–action rule**, written as.

if car-in-front-is-braking then initiate-braking.



Schematic diagram of a simple reflex agent

By, Dr. Keerthan, ISE, SIT, TUMKUR

Simple Reflex Agents

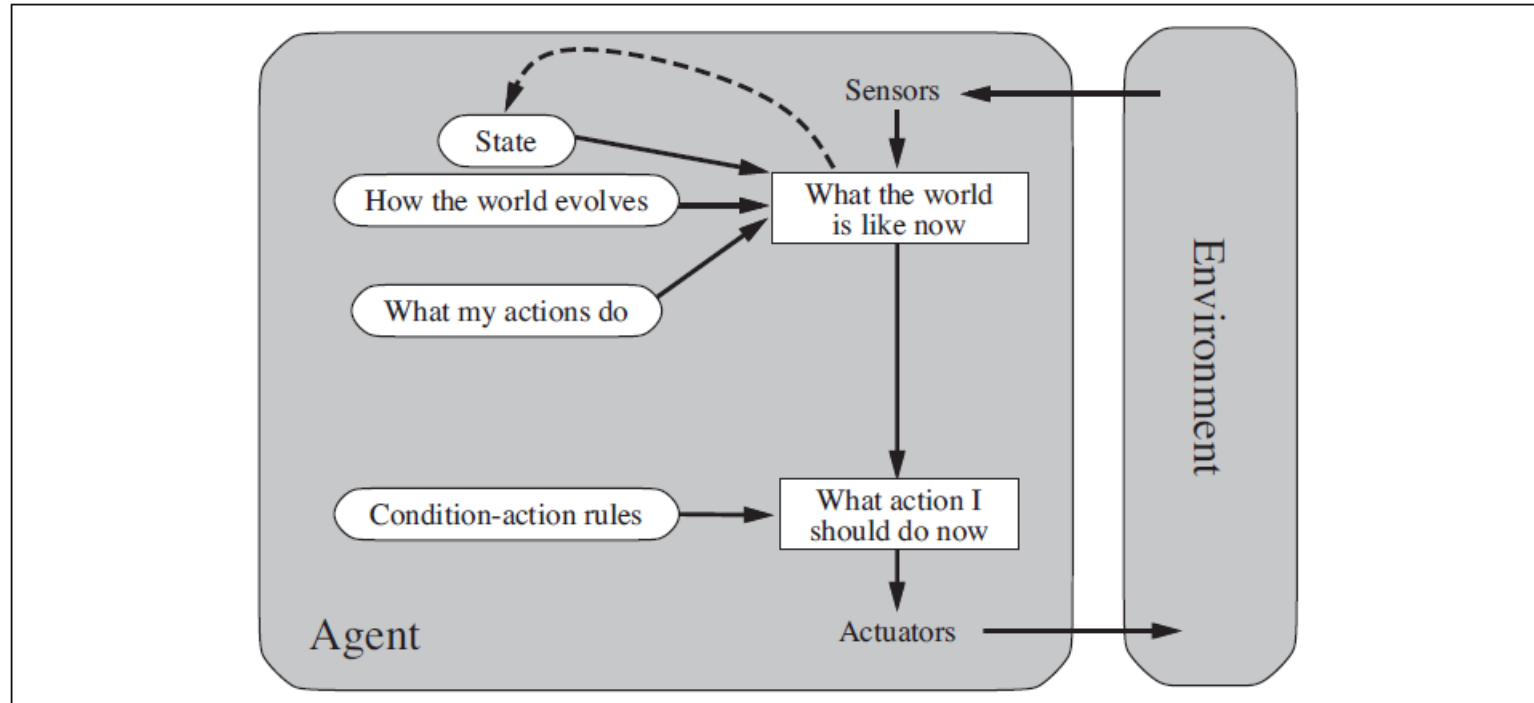
```
function SIMPLE-REFLEX-AGENT(percept) returns an action  
  persistent: rules, a set of condition–action rules  
  
  state ← INTERPRET-INPUT(percept)  
  rule ← RULE-MATCH(state, rules)  
  action ← rule.ACTION  
  return action
```

- A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

The Structure of Agents contd.

- **Model-Based Reflex Agents:** These agents maintain an internal model of the world, allowing them to keep track of parts of the environment that are not directly observable. They base actions not only on current perceptions but also on the internal state.
- **Example:** A robot vacuum that remembers areas it has cleaned.
- **The most effective way to handle partial observability is for the agent to keep track of the part of the world it can't see now.**
- **i.e., the agent should maintain some sort of internal state that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state.**

Model-Based Reflex Agents contd.



A model-based reflex agent

Model-Based Reflex Agents contd.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends on current state and action
               rules, a set of condition–action rules
               action, the most recent action, initially none

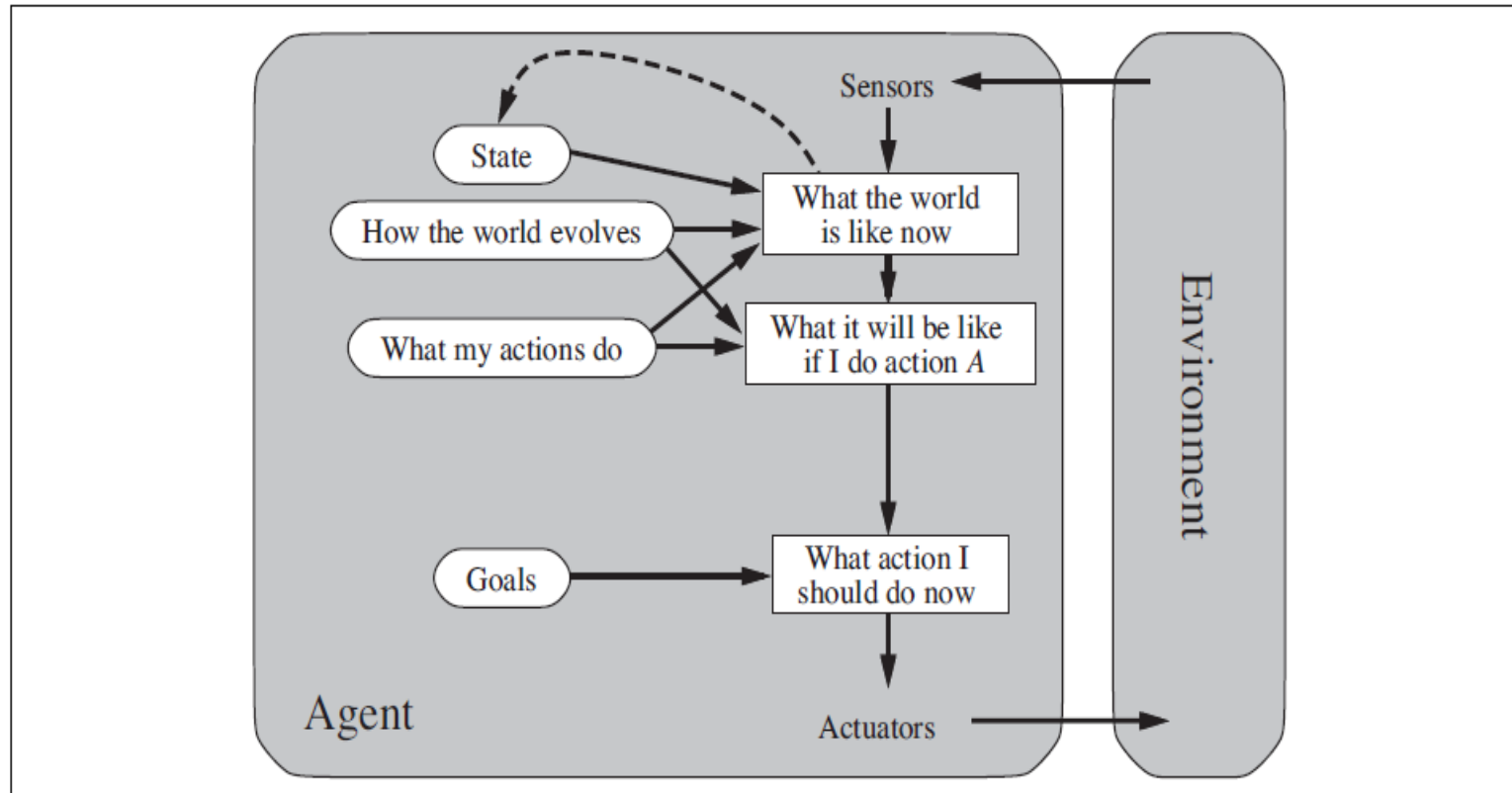
  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Goal-Based Agents

- **Goal-Based Agents:** In addition to tracking the environment, these agents are driven by specific goals.
- They choose actions that will help them achieve their goals, requiring planning and decision-making.
- **Example:** A navigation system that selects routes based on the goal of reaching a destination.

Goal-Based Agents

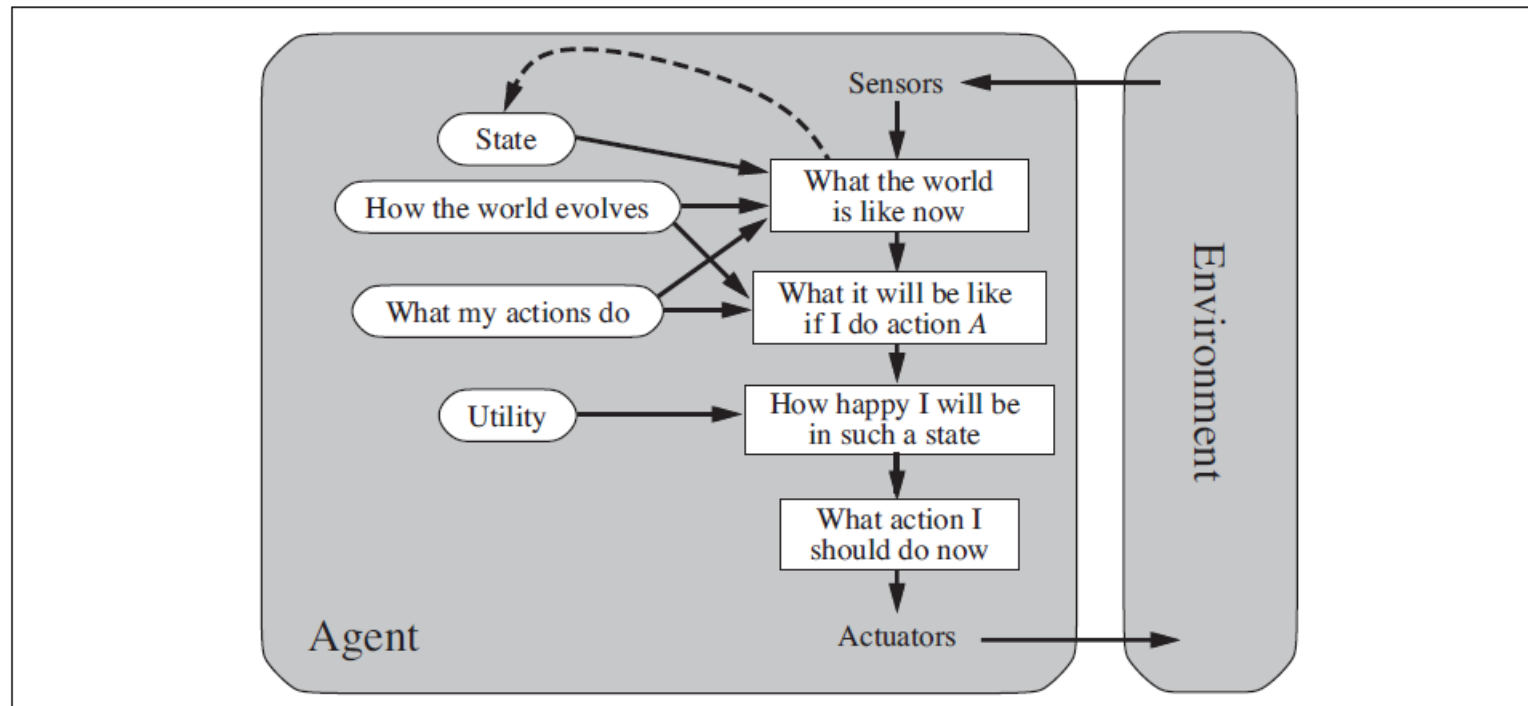


A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Utility-Based Agents.

- **Utility-Based Agents:**
- Goals alone are not enough to generate high-quality behavior in most environments.
- For example, many action sequences will get the taxi to its destination thereby achieving the goal.
- But some are quicker, safer, more reliable, or cheaper than others.
- Goals just provide a crude binary distinction between “happy” and “unhappy” states.
- A **goal** just says what you want to achieve, not *how well* you achieve it.
- An agent’s utility function is essentially an internalization of the performance measure..
- These agents aim to maximize their performance by choosing actions based on a utility function, which ranks states according to their desirability.
- **Example:** An autonomous trading agent that selects trades based on potential profits.

Utility-Based Agents contd.

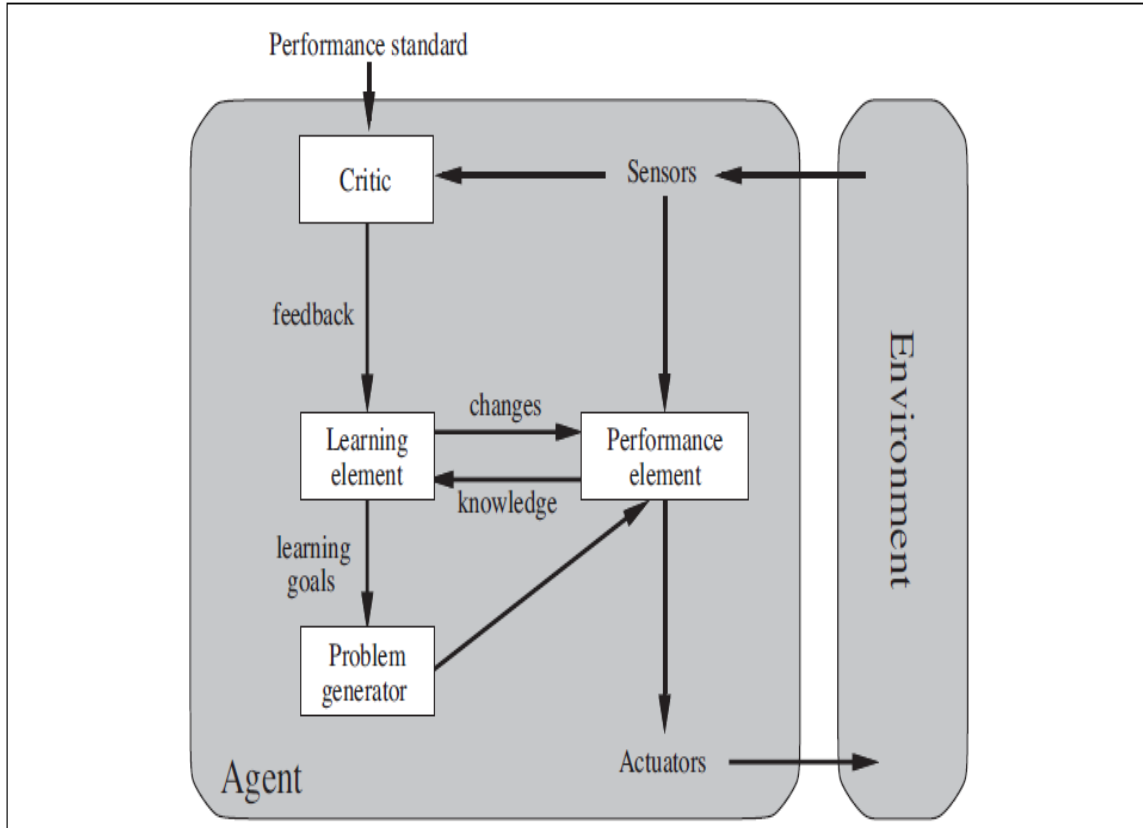


A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning Agents:

- **Learning Agents:** These agents can improve their performance over time by learning from their experiences. They adjust their behavior based on feedback or outcomes, enhancing their decision-making process.
- **Example:** A recommendation system that improves suggestions based on user preferences over time.

Learning Agents contd.



A general learning agent.

A learning agent can be divided into four conceptual components.

Learning element, which is responsible for making improvements, and the **performance element**, which is responsible for selecting external actions.

The learning element uses feedback from the **critic** on how the agent is doing and determines how the performance element should be modified to do better in the future.

the **problem generator** is responsible for suggesting actions that will lead to new and informative experiences.

If the agent is willing to explore a little and do some perhaps suboptimal actions in the short run, it might discover much better actions for the long run. The problem generator's job is to suggest these exploratory actions

Example: Self-driving taxi

- **Performance element:** Always takes the *known shortest route* to the destination.
- But maybe there's another route that seems longer, but is actually *faster during traffic hours*.
- **Problem generator:** Suggests “Try this new route once” → even if it's not optimal right now.
- If it works better, the **learning element** updates the knowledge, and the taxi becomes smarter for future trips.

SOLVING PROBLEMS BY SEARCHING

- A **problem-solving agent** is an intelligent agent that uses search and planning to find a sequence of actions that leads to a specific goal.
- These agents are designed to solve problems by *reasoning through potential solutions* and *selecting the most appropriate course of action*.
- Here are the **key components and characteristics** of problem-solving agents:
- **Goal Formulation:**
 - The agent begins by **clearly defining its goal**, which is the desired *outcome*.
 - The *goal provides* a criterion for success and allows the agent to determine whether it has *achieved the objective*.
 - **Example:** In a *navigation task*, the goal could be reaching a specific location (destination).

SOLVING PROBLEMS BY SEARCHING

- **Problem Formulation:**

- The agent must formulate the problem it needs to solve.
- This involves defining the possible states, actions, transition model, and the goal state.
- i.e, The problem formulation stage sets the groundwork for search.
- **Example:** For the 8-puzzle problem, the formulation involves defining the possible tile configurations and allowable moves.

SOLVING PROBLEMS BY SEARCHING

- **Search:**

- The process of looking for a sequence of actions that reaches the goal is called search.
- A search algorithm takes a problem as input and returns a solution in the form of an action sequence.
- **Example:** In the 8-puzzle problem, the agent uses search to find a sequence of moves (up, down, left, right) to arrange the tiles in the correct order.

SOLVING PROBLEMS BY SEARCHING

- **Solution Execution:**

- Once a solution is found (i.e., a sequence of actions leading to the goal) the **agent executes the actions step by step.**
- The environment changes based on the agent's actions, and it continues until the goal is reached.
- It is an important property that in a fully observable, deterministic, known environment, the solution to any problem is a fixed sequence of actions.
- **Example:** In a robot navigation task, the robot follows the planned path to the destination.

SOLVING PROBLEMS BY SEARCHING

- The open-loop system means that ignoring the percepts breaks the loop between agent and environment.
 - In an open-loop system, the agent operates without using feedback from its environment while executing its actions.
 - This means that once it starts following a plan or sequence of actions, it does so without adjusting based on real-time percepts or sensory input.
 - Essentially, it "ignores" its current environment because it assumes that everything will go as predicted".
- If there is a chance that the model is incorrect, or the environment is nondeterministic, then the agent would be safer using a closed-loop approach that monitors the percepts.
 - Constantly monitors the environment (using percepts or feedback) and adjusts actions accordingly.
 - Closed-loop systems are typically more robust in dynamic or unpredictable environments because they can adapt to new information.

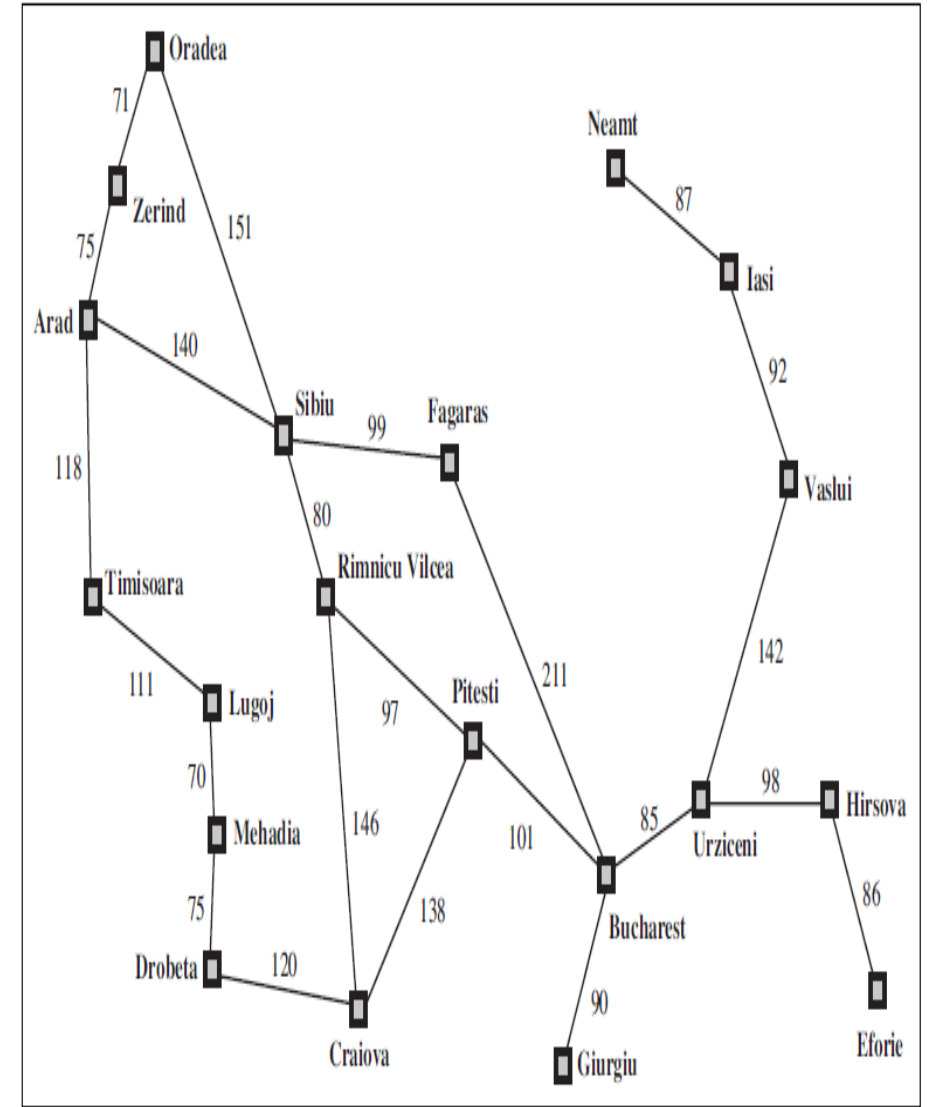
Problem-Solving Process

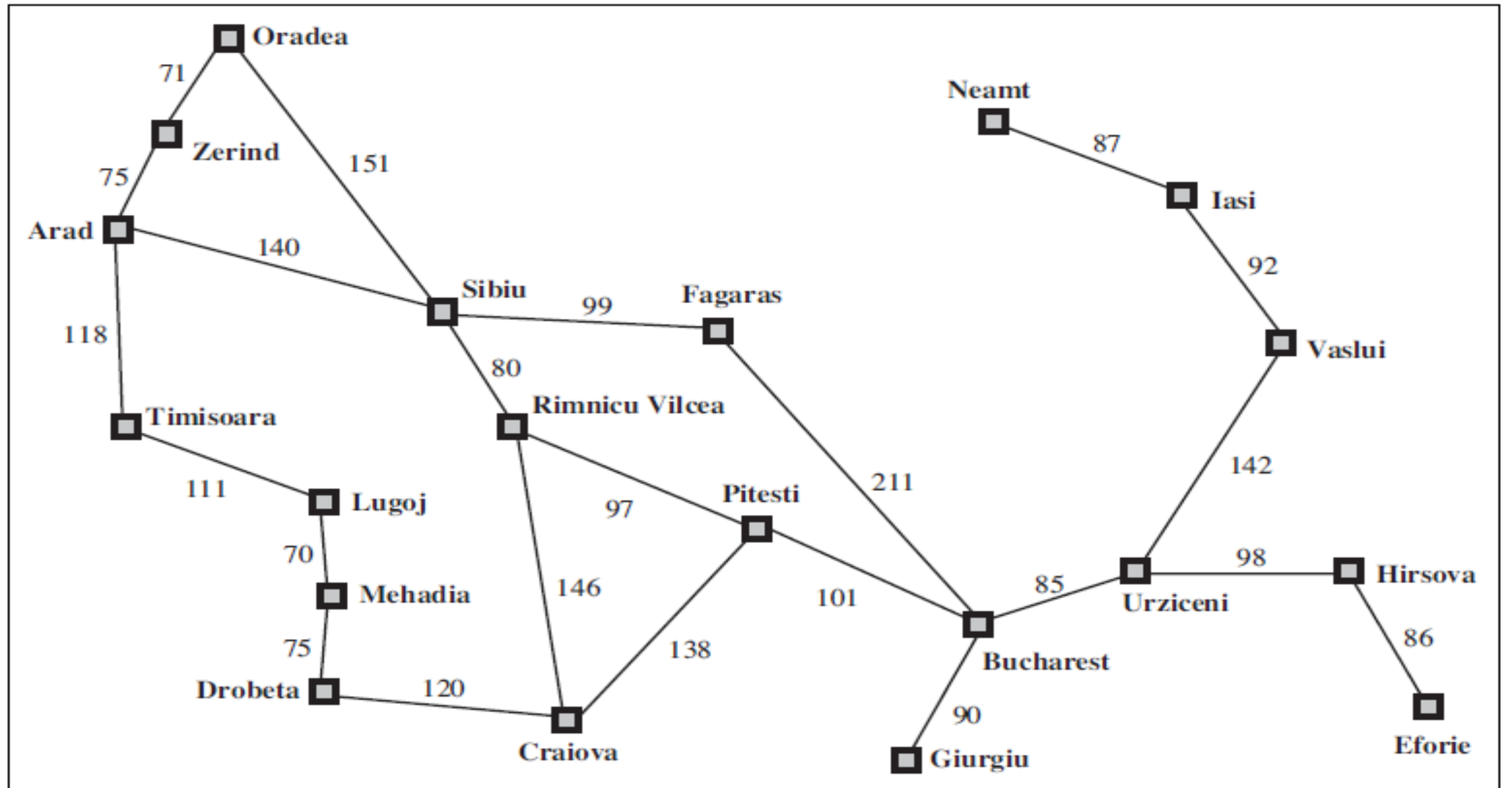
- A problem-solving agent typically follows these steps in solving a problem:
 - 1. Define the Goal:** The agent determines what it needs to achieve.
 - 2. Formulate the Problem:** The agent identifies the initial state, goal state, actions, and transition model.
 - 3. Search for a Solution:** The agent searches the problem space to find a sequence of actions that leads to the goal state.
 - 4. Execute the Solution:** Once the sequence is found, the agent performs the actions in order to achieve the goal.

Well-defined problems and solutions.

A **problem** can be defined formally by five components:

1. **Initial State:** The initial **state** that the agent starts in. For example, the initial state for our agent in Romania might be described as **In(Arad)**.
2. A description of the possible **actions** available to the agent. Given a particular state **s**, **ACTIONS(s)** returns the set of actions that can be executed in **s**. We say that each of these actions is applicable in **s**. For example, from the state **In(Arad)**, the applicable actions are **{Go(Sibiu), Go(Timisoara), Go(Zerind)}**.

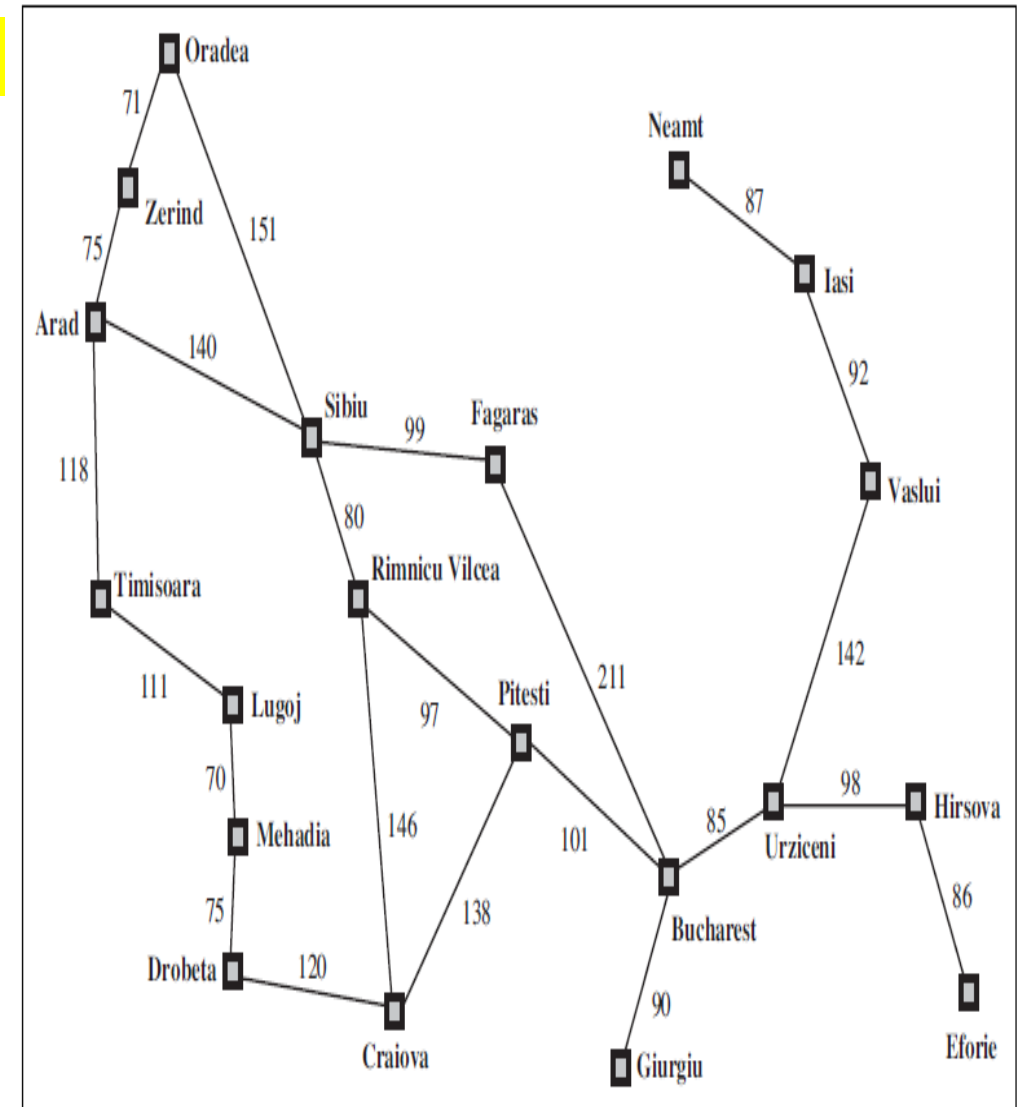




Well-defined problems and solutions.

3. A description of what each action does; the formal name for this is the **transition model**, specified by a function **RESULT(s, a)** that returns the state that results from doing action **a** in state **s**. We also use the term **successor** to refer to any state reachable from a given state by a single action. For example, we have **RESULT(In(Arad),Go(Zerind)) = In(Zerind)**.

- Together, the initial state, actions, and transition model implicitly define the **state space** of the problem—the set of all states reachable from the initial state by any sequence of actions.



Well-defined problems and solutions.

- The **state space forms a directed network or graph** in which the nodes are states and the links between nodes are actions.
- A **path** in the state space is a sequence of states connected by a sequence of actions.
- **4. The goal test:** which determines whether a given state is a goal state. The agent's goal in Romania is the singleton set {In(Bucharest)}.
- **5. A path cost *function that assigns a numeric cost to each path***. The problem-solving agent chooses a cost function that reflects its own performance measure.
- The **step cost of taking action a in state s to reach state s** is denoted by $c(s, a, s)$. The step costs for Romania are shown in Figure 3.2 as route distances. We assume that step costs are nonnegative.

Contd.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state  $\leftarrow$  UPDATE-STATE(state, percept)
  if seq is empty then
    goal  $\leftarrow$  FORMULATE-GOAL(state)
    problem  $\leftarrow$  FORMULATE-PROBLEM(state, goal)
    seq  $\leftarrow$  SEARCH(problem)
    if seq = failure then return a null action
  action  $\leftarrow$  FIRST(seq)
  seq  $\leftarrow$  REST(seq)
  return action
```

- A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

The **vacuum world**. This can be formulated as a problem as follows:

States: The state is determined by both **the agent location and the dirt locations**. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.

- **Initial state:** Any state can be designated as the initial state.
- **Actions:** In this simple environment, each state has just three actions: *Left*, *Right*, and *Suck*. Larger environments might also include *Up* and *Down*.
- **Transition model:** The actions have their expected effects, except that moving *Left* in the leftmost square, moving *Right* in the rightmost square, and *Sucking* in a clean square have no effect. The complete state space is shown in Figure 3.3.
- **Goal test:** This checks whether **all the squares are clean**.
- **Path cost:** Each step costs 1, so **the path cost is the number of steps in the path**.

The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- Toy problems

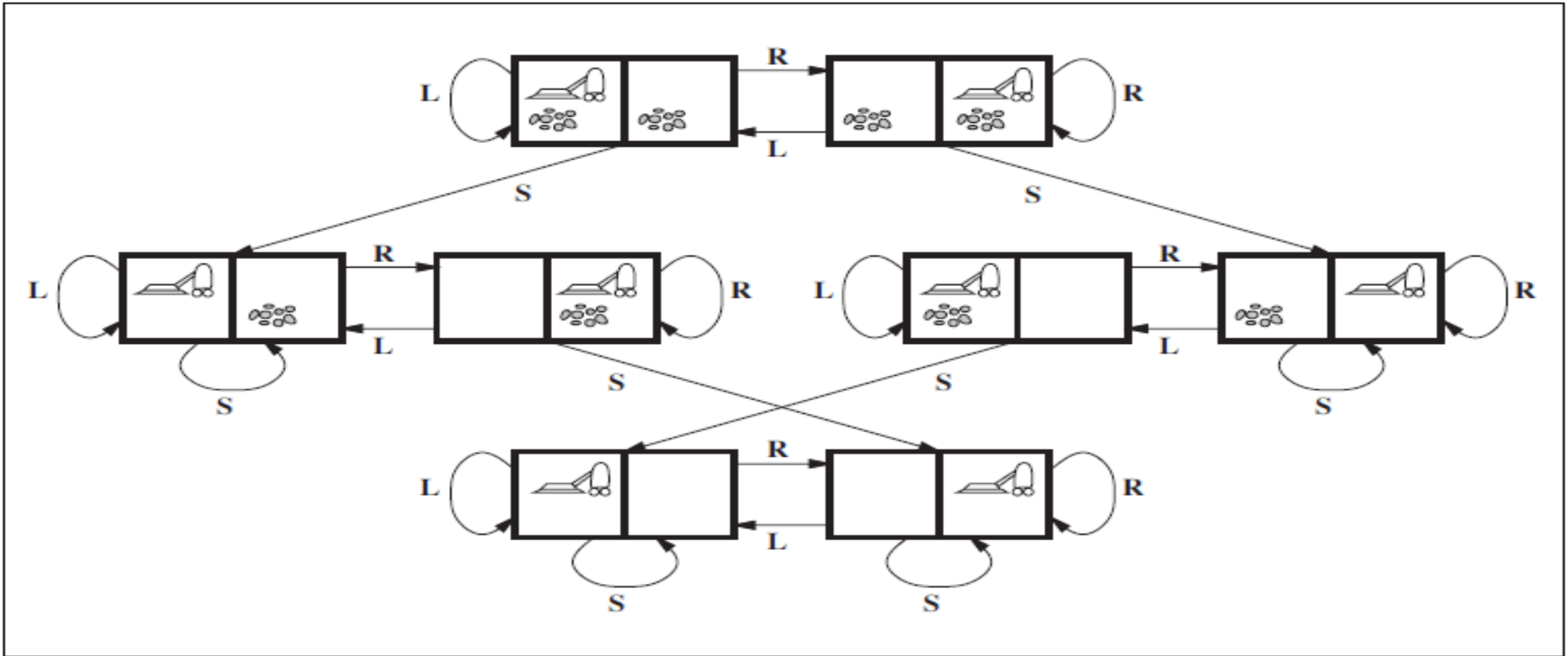
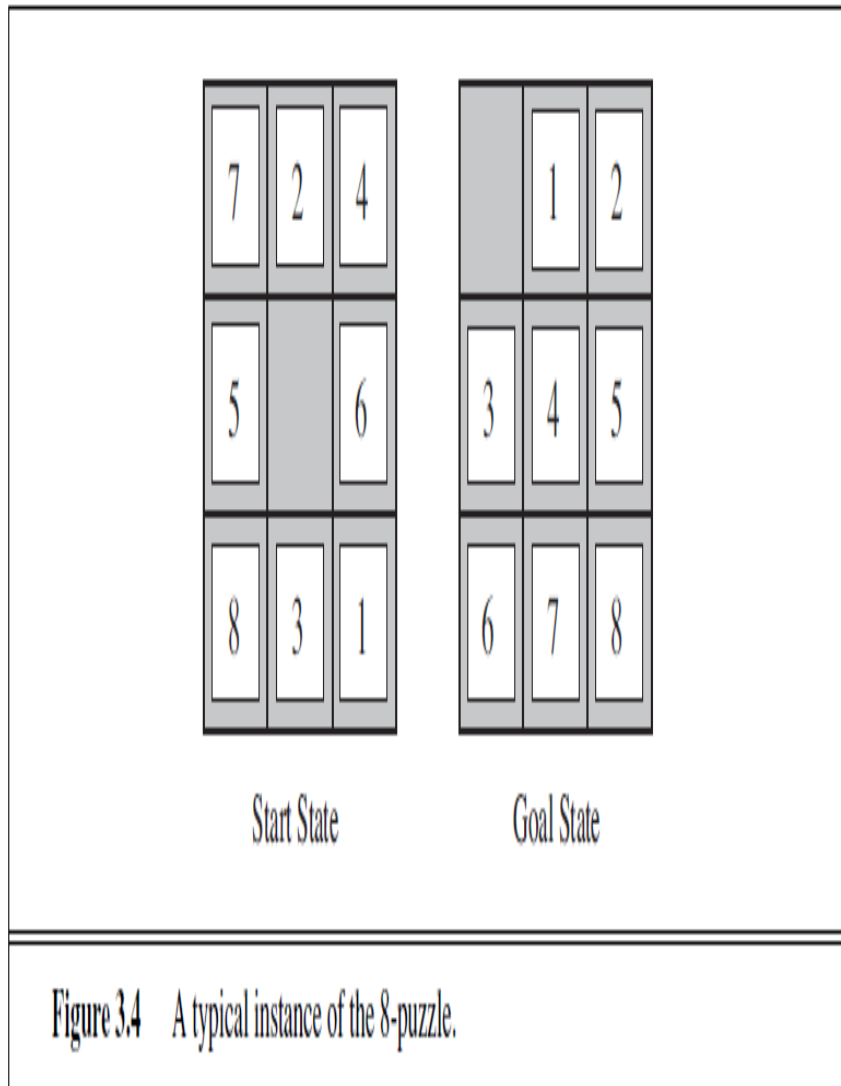


Figure 3.3 The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

The **8-puzzle**, an instance of which is shown in Figure 3.4, consists of a 3×3 board with eight numbered tiles and a blank space. A tile adjacent to the blank space can slide into the space. The object is to reach a specified goal state, such as the one shown on the right of the figure. The standard formulation is as follows:



- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state:** Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4).
- **Actions:** The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.

-

The 8-puzzle, contd.

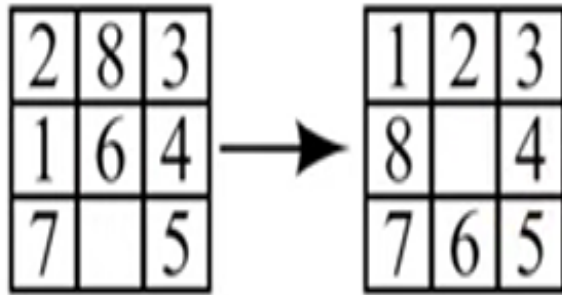


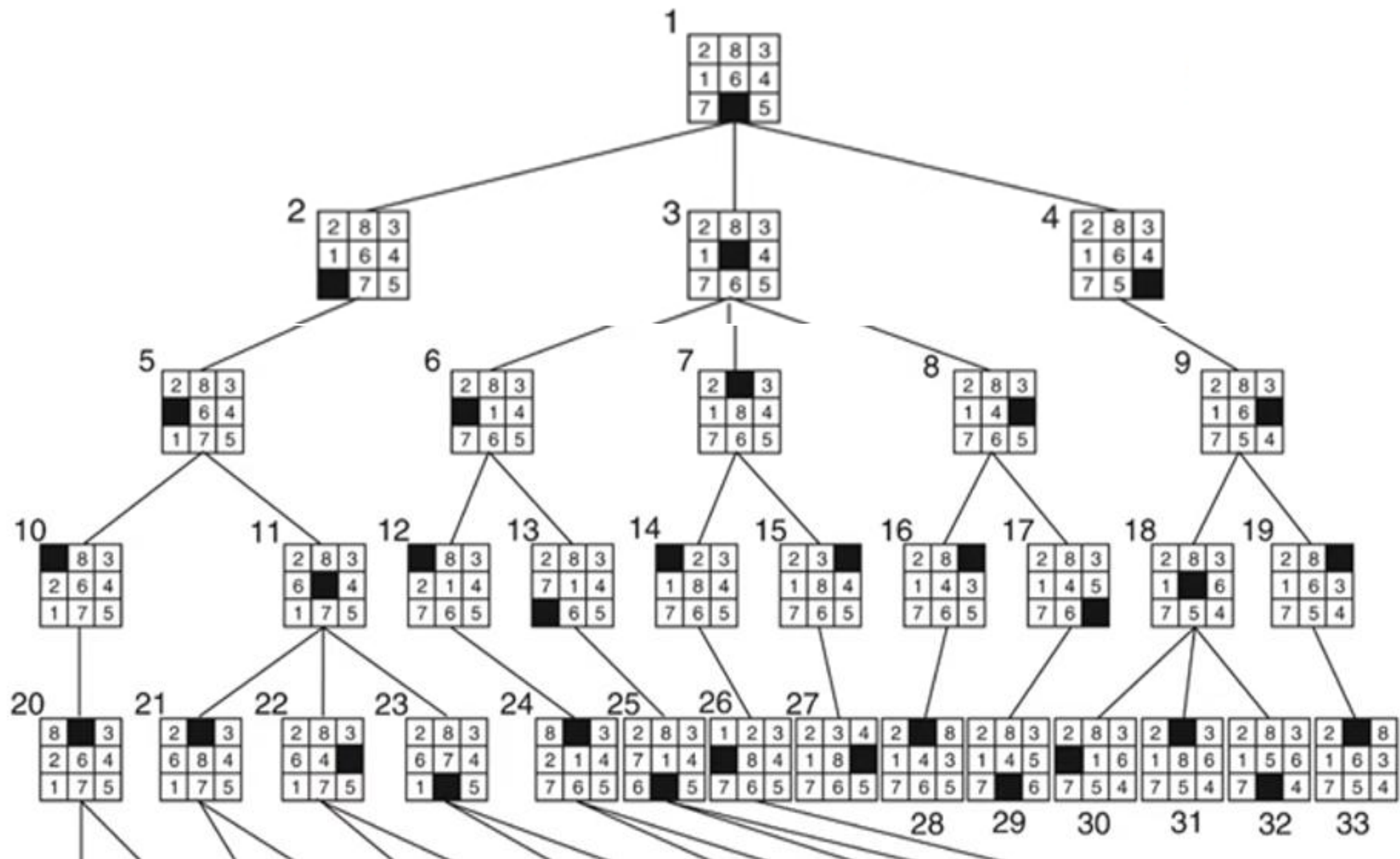
Start State

Goal State

- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
- **Goal test:** This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

Figure 3.4 A typical instance of the 8-puzzle.





8	3		8	6	3		2	3	2	3		2	8		2	8	3	2	8	3	2	8	3	2	8	3	8	3		8	1	3	2	8	3	2	8	3	1	2	3	
2	6	4	2		4	6	8	4	6	8	4	6	4	3	6	4	5	6	7	4	6	7	4	2	1	4	2		4	7		4	7		4	7	1	4	8		4	
1	7	5	1	7	5	1	7	5	1	7	5	1	7	5	1	7			1	5	1	5		7	6	5	7	6	5	7	6	5	6	1	5	6	5		7	6	5	
34	35	36	37	38	39	40	41	42	43	44	45	46																														

Solve the following 8 puzzle problem with heuristic approach and also write solution space tree.

7	2	4
5		6
8	3	1
Start State		

	1	2
3	4	5
6	7	8
Goal state		

8-queens problem

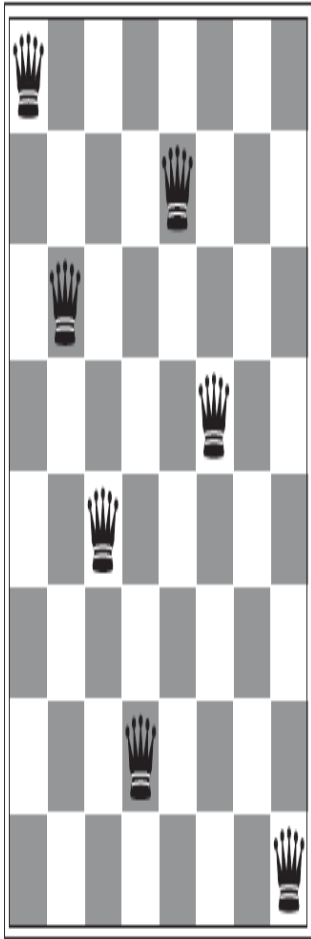


Figure 3.5 Almost a solution to the 8-queens problem. (Solution is left as an exercise.)

- Initial State:** An empty **8x8 chessboard with no queens placed on it.** Alternatively, the initial state can start with a partially filled board, depending on the solution approach (like backtracking or local search).

- Goal State:** A configuration of the chessboard where 8 queens are placed such that **no two queens threaten each other.** This means no two queens share the same row, column, or diagonal.

- Actions:** **Placing a queen in a specific row and column on the chessboard in such a way that it doesn't conflict with the positions of other queens already on the board.** Generally, solutions restrict actions to placing one queen per row or column to reduce complexity.

- Transition Model:** **The rules governing valid placements of queens.** For instance, when placing a queen, the model checks for conflicts with other queens already placed (in rows, columns, and diagonals) to ensure each placement follows the problem's constraints.

- Path Cost:** For the 8-queens problem, path cost is typically not defined, as it is often formulated as a constraint satisfaction problem (CSP) or solved through a local search.

Instead, the objective is to simply find a solution that meets all constraints without needing to minimize a path cost.

Real-world problems

- Route-finding algorithms are used in a variety of applications. Some, such as Web sites and in-car systems that provide driving directions, are relatively straightforward extensions of the Romania example. Others, such as routing video streams in computer networks, military operations planning, and airline travel-planning systems, involve much more complex specifications.

- Consider the airline travel problems that must be solved by a travel-planning Web site

States: Each state obviously includes a location (e.g., an airport) and the current time. Furthermore, because the cost of an action (a flight segment) may depend on previous segments, their fare bases, and their status as domestic or international, the state must record extra information about these “historical” aspects.

Initial state: This is specified by the user’s query.

Actions: Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.

Transition model: The state resulting from taking a flight will have the flight’s destination as the current location and the flight’s arrival time as the current time.

Goal test: Are we at the final destination specified by the user?

Path cost: This depends on monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, and so on.

- **Initial State:** The initial state includes all user-provided information, such as:
 - Departure and destination locations
 - Departure and return dates (or a flexible date range)
 - Budget constraints
 - Preferences for airlines, class of service, layovers, and additional options (e.g., baggage, in-flight meals)
 - Any previous travel history or user profile information that might influence recommendations
- **Actions:** The possible actions available to the travel-planning system, including:
 - Searching for available flights that match user criteria
 - Filtering or sorting results based on cost, duration, layovers, etc.
 - Modifying search parameters to offer alternative options (e.g., suggesting nearby airports or flexible dates)
 - Offering add-ons like insurance, seat selection, or accommodations
- **Transition Model:** The transition model defines how each action affects the state of the system.
For example:
 - Selecting a flight or modifying search criteria updates the system's state, refining or changing the list of available flights.
 - Adjusting budget or preferences leads to new search results.
 - Selecting a flight option transitions to a booking state, where the user can finalize their choice.

- **Goal State:** The goal state is achieved when the system finds a solution that meets the user's criteria and preferences. This could mean:
 - Presenting the user with a flight option that aligns with all specified preferences and constraints
 - Suggesting a close alternative if no exact match is found (e.g., a slightly more expensive or longer flight)
 - Allowing the user to complete the booking, moving from the search state to the confirmation and payment state
- **Cost Function:** The cost function evaluates the “expense” or “effort” of each action or choice, aiming to find the most optimal result. Cost factors could include:
 - Ticket price, which should align with the user's budget
 - Travel duration, including layovers and total journey time
 - Convenience factors, such as departure and arrival times, and airport locations
 - User satisfaction, factoring in historical preferences, loyalty to specific airlines, or other personal preferences

For a Touring Problem

1.Initial State:

The initial state is the starting location of the tour (e.g., the user's home city or a designated starting point). It includes information such as:

1. A list of all destinations or cities that need to be visited
2. Any known constraints, such as time limits or budget
3. The current location within the tour

2.Actions:

The actions in a touring problem represent the choices for moving from one location to another. These actions could include:

1. Traveling from the current location to the next destination on the list
2. Selecting an optimal route between destinations based on distance, travel time, or cost
3. Rearranging or skipping certain destinations if flexibility is allowed

3.Transition Model:

The transition model defines how each action (moving to a new location) changes the state of the system. For instance:

1. Traveling from one destination to another updates the current location and reduces the list of remaining destinations
2. Changes in state may also update the remaining time or budget, depending on the travel cost and time for each leg of the tour

COND.

4. Goal State:

The goal state is achieved when the traveler has visited all required destinations and returned to the starting point (if it's a round trip) or reached a designated final destination. Specifically:

1. All cities on the list have been visited exactly once (or as required)
2. The route taken minimizes cost or travel time according to user preferences
3. Constraints (like budget and time limits) are respected, and the tour is completed within those bounds

5. Cost Function:

The cost function assesses the total "expense" of a given tour route, which can be optimized based on:

1. Total travel distance or time, aiming to minimize it
2. Total travel cost, especially if there's a budget constraint
3. Other user-defined preferences, such as scenic value, convenience, or preferred modes of travel
4. Penalties for missing destinations, if the user is unable to complete the full tour

Traveling Salesperson Problem (TSP)

•Initial State:

The initial state is the starting city (or node) where the salesperson begins their journey. This initial state includes:

- The list of all cities or locations that need to be visited
- The current city (initially the starting city)

•Actions:

The actions available in TSP involve choosing the next city to visit from the current location. These actions are defined as:

- Traveling from the current city to any other unvisited city
- Marking the chosen city as "visited" to avoid revisiting it within the same route

Transition Model:

The transition model describes how each action (traveling to a new city) affects the state:

- Moving to a new city updates the current location and removes that city from the list of unvisited cities
- This transition also accumulates the distance or cost of traveling to that city, contributing to the total route cost

Contd.

- Goal State:**

The goal state is achieved when the salesperson has:

- Visited each city exactly once
- Returned to the starting city to complete the route
- Minimized the total travel distance or cost

- Cost Function:**

The cost function evaluates the total cost of the route based on:

- The cumulative distance or cost of the entire route
- The objective is to minimize this total cost, finding the shortest possible route that visits all cities once and returns to the starting point

Automatic Assembly Sequencing

- **Initial State:**

The initial state is the starting point of the assembly process, where:

- All parts are unassembled
- All assembly resources (e.g., machines, tools, and workers) are available
- No steps in the assembly sequence have been completed yet

- **Actions:**

The actions in assembly sequencing involve choosing the next assembly step or operation. Each action corresponds to:

- Attaching or assembling a specific part to the growing product according to design requirements
- Performing operations in an order that respects dependencies (e.g., some parts can only be assembled after others are in place)

- **Transition Model:**

The transition model describes how each action (performing an assembly step) changes the state:

- After each assembly step, the system updates the state by marking the completed part or subassembly as "assembled"
- Constraints such as dependencies between parts or subassemblies are adjusted accordingly, enabling other dependent steps

Cntd.

•Goal

State:

The goal state is achieved when:

- All parts have been successfully assembled in the correct order
- The final product is fully assembled **without any violations of dependencies or constraints**
- The assembly is completed in the most efficient manner possible (minimizing time, cost, or resource usage)

•Cost

Function:

The cost function measures **the efficiency of the assembly sequence**, which can be **optimized** based on:

- Total assembly time, aiming **to minimize the overall time** required to complete the product
- Resource usage, including machinery, tools, and labor, with the goal of **minimizing costs associated with these resources**
- Avoidance of **unnecessary or redundant assembly steps**
- Adherence to quality standards**, ensuring the product is assembled correctly and reliably

- **Thank you.....**